



Configuration d'un environnement de développement respectant la chaine DevOps

-Système d'exploitation & Linux-

Réalisé par:

-DAHBI AYA
-BOUFNAR DOUAE
-ELASRI FATMA ZAHRA
-ZAKARIA REDA
-OUGGOURNI AYMEN

Encadré par:

Mr. EL BOUHDIDI JABER

Tuteur :

EL ALLOUCHE ZAKARIYAE

Table des matières

1. Remerciements
2. Introduction

Chapitre 1 : Organisation de l'équipe et gestion du projet

- Répartition des tâches techniques
- Liaison entre GitHub et Jira
- Désignation de la cheffe de projet
- Responsables du développement
- Responsables GitHub et déploiement
- Responsables de la configuration d'environnement

Chapitre 2 : Choix technologiques

- Linux
- Docker
- Jira
- GitHub
- Terraform
- Laravel
- Angular
- Nginx
- AWS

Chapitre 3 : Mise en place de l'environnement Docker Installation et configuration

- Installation de Docker sur Ubuntu
- Configuration de Docker Hub

Création et gestion des images Docker

- Dockerfile MySQL
- Fichier init.sql
- Fichier my.cnf
- Build de l'image MySQL
- Dockerfile Laravel
- Configuration Nginx
- Build de l'image Laravel
- Dockerfile Angular
- Build de l'image Angular

Docker Compose

- Structure du fichier docker-compose.yml
- Configuration du docker-compose du projet
- Build et exécution des conteneurs

Chapitre 4 : Intégration Continue et Tests (CI)

- Utilisation de GitHub et GitHub Actions
- Configuration des secrets GitHub
- Structure du pipeline CI
- Tests d'authentification Backend
- Tests de gestion des livres
- Validation automatisée des fonctionnalités

Chapitre 5 : Utilisation de Terraform et Infrastructure AWS

- Installation de Terraform
- Structure du projet Terraform
- Fichier main.tf
- Fichier variables.tf
- Fichier security.tf
- Fichier outputs.tf
- Déploiement de l'infrastructure
- Connexion SSH à l'instance EC2

Chapitre 6 : Pipeline CD — Déploiement Continu

- Introduction au pipeline CD
- Déclenchement du pipeline
- Build et Push des images Docker
- Déploiement automatique sur AWS
- Démonstration du déploiement
- Bénéfices du pipeline CD

Chapitre 7 : Difficultés rencontrées

- Problèmes liés aux workflows GitHub Actions
- Difficultés de déploiement Docker
- Gestion des permissions Laravel
- Conflits GitHub et fusion des branches

Chapitre 8 : Retour d'expérience

- Retour d'expérience de l'équipe
- Compétences acquises
- Expérience DevOps et travail collaboratif

Conclusion

Annexes

Introduction

Dans le cadre du mini-projet de Système d'Exploitation réalisé au sein de la filière Génie Informatique 1 à l'ENSA de Tétouan pour l'année universitaire 2025/2026, nous avons été amenés à concevoir un environnement de développement web collaboratif basé sur les technologies DevOps modernes. Ce projet a pour objectif principal de mettre en place une plateforme de développement standardisée permettant à tous les membres de l'équipe de travailler dans des conditions identiques, tout en facilitant la collaboration, le partage du code et le déploiement de l'application.

Dans un contexte de développement collaboratif, les différences entre les environnements de travail des développeurs représentent souvent une source importante de problèmes techniques. En effet, chaque membre peut utiliser un système d'exploitation différent (Windows, Linux ou macOS), ainsi que des versions variées des frameworks et outils de développement comme PHP, Laravel ou Angular. Cette hétérogénéité peut provoquer des conflits de dépendances, des erreurs d'exécution et des difficultés lors de l'intégration ou du déploiement du projet.

Afin de résoudre cette problématique, Docker a été utilisé comme solution principale pour standardiser les environnements de développement. Grâce aux conteneurs Docker, l'ensemble des dépendances, configurations et services nécessaires au fonctionnement de l'application sont regroupés dans une image unique et reproductible. Ainsi, tous les membres de l'équipe travaillent dans un environnement homogène, ce qui améliore la cohérence du développement et réduit considérablement les erreurs liées aux différences de configuration.

En parallèle, plusieurs outils modernes ont été intégrés afin de reproduire un véritable environnement de travail professionnel. GitHub a été utilisé pour assurer le versionnement et la gestion collaborative du code source, tandis que Jira/Notion a permis d'organiser les tâches, suivre l'avancement du projet et améliorer la communication entre les membres de l'équipe. De plus, Terraform a été exploité dans le but d'automatiser la création et la configuration d'un serveur Linux destiné au déploiement de l'application, conformément aux principes de l'Infrastructure as Code (IaC).

Ce projet nous a permis non seulement d'approfondir nos connaissances techniques dans les domaines des systèmes d'exploitation, de Docker et des outils DevOps, mais également de développer des compétences essentielles telles que le travail en équipe, la gestion de projet, la collaboration et l'organisation dans un cadre proche du monde professionnel.

Chapitre 1 : Répartition des tache techniques et liaison de github et jira

On a connecté Jira à notre repo GitHub: chaque branche créée porte le nom du ticket , les commits sont liés automatiquement au ticket avec la référence .

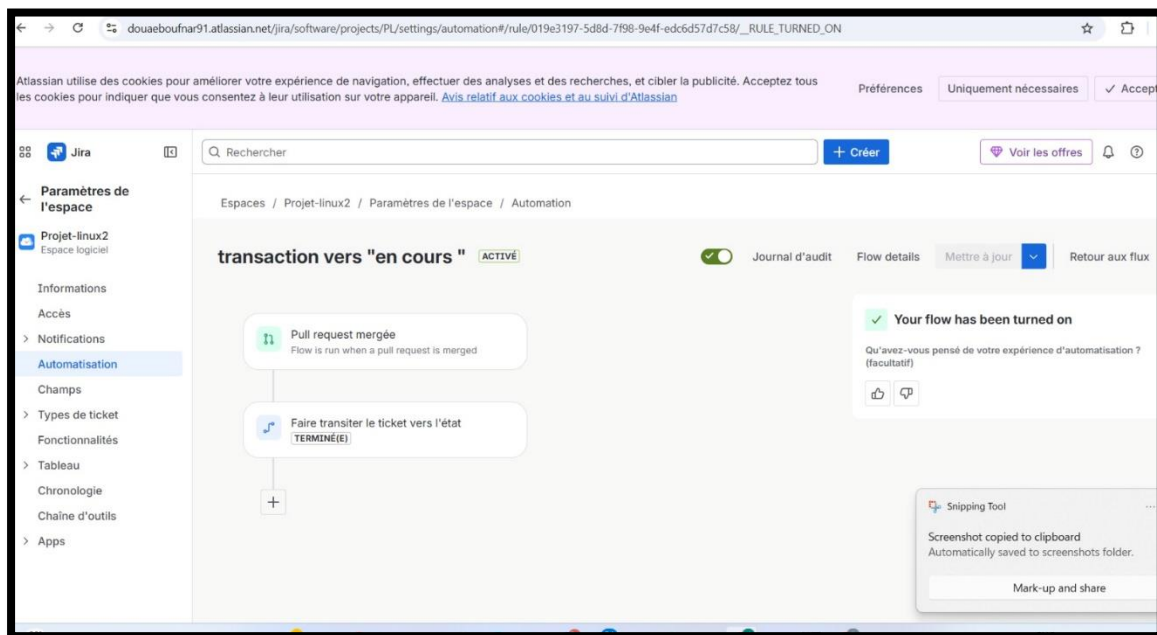


Figure 1-1-initialisation de jira

La figure suivante montre que la connexion de github avec jira était faite avec succès

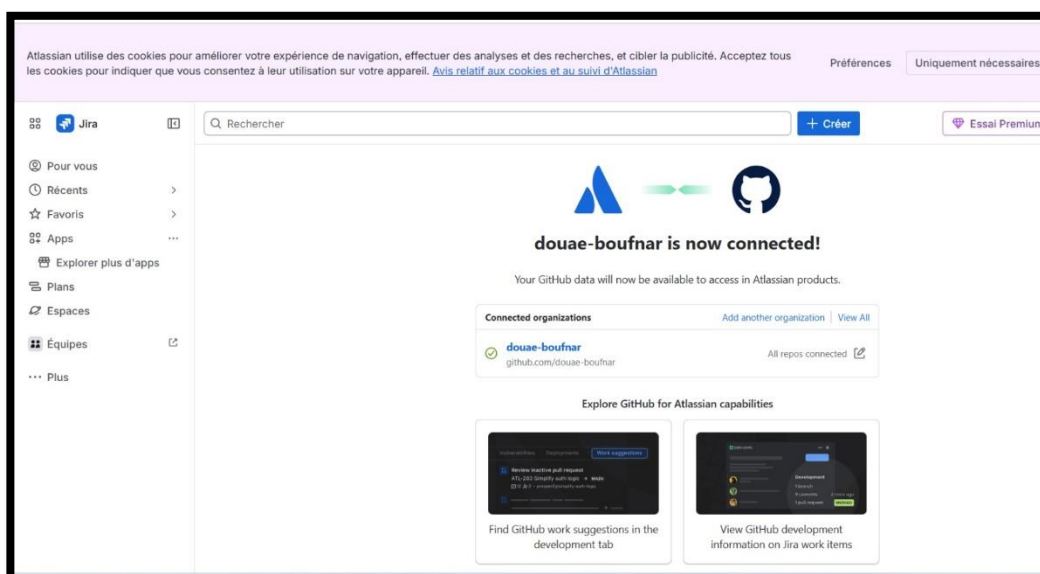


Figure 1-2-connexion de Jira avec Github

- *Répartition des rôles techniques*

- **Désignation d'une cheffe de projet :**

- La cheffe du projet a été désignée afin d'assurer la coordination générale de l'équipe. Elle a eu pour mission de configurer et gérer Jira, organiser les différentes tâches du projet et suivre l'avancement du travail tout au long du développement. Elle a également participé à la configuration de l'environnement Docker afin de garantir un environnement de travail stable et identique pour tous les membres de l'équipe. Son rôle consistait aussi à faciliter la communication entre les membres et à assurer le bon déroulement du projet.

- Responsables du Développement :**

- L'équipe de développement a participé à la réalisation des parties Front-end et Back-end de l'application. Elle a assuré l'implémentation des différentes fonctionnalités, l'intégration entre les composantes du système ainsi que la résolution des problèmes techniques rencontrés durant le projet.

- Certains membres ont également participé à la configuration de Terraform afin d'automatiser l'infrastructure et le déploiement de l'application sur le serveur Linux.

- L'équipe a veillé à maintenir une bonne organisation du code et à garantir la cohérence technique entre les différentes parties du projet.

- **Responsables GitHub et Déploiement :**

- L'équipe responsable de GitHub et du déploiement a assuré la gestion du dépôt GitHub du projet ainsi que le bon versionnement du code source. Elle s'est chargée de l'organisation des branches, du suivi des pull requests et de la collaboration entre les membres de l'équipe.

- Elle a également participé à la configuration de Terraform ainsi qu'à la mise en place de GitHub Actions afin d'automatiser certaines tâches liées à l'intégration continue et au déploiement du projet.

- Responsables de la Configuration d'Environnement :

- L'équipe responsable de la configuration de l'environnement a participé à la création et à la configuration de l'environnement Docker utilisé durant le développement du projet. Cela inclut la mise en place des conteneurs nécessaires ainsi que la standardisation de l'environnement de travail pour tous les membres de l'équipe.
- L'objectif principal était de garantir la stabilité, la compatibilité et la cohérence du projet entre les différentes machines utilisées pendant les phases de développement, de test et de déploiement.

Chapitre 2 : Choix Technologique

• *Linux :*

Le système d'exploitation Linux a joué un rôle essentiel dans notre projet, notamment pour l'exécution et le déploiement de l'application.

Grâce à sa stabilité, sa sécurité et sa compatibilité avec les outils DevOps modernes, Linux constitue un environnement idéal pour l'hébergement d'applications web.

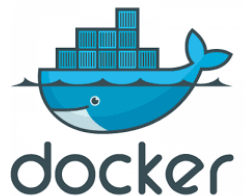
Il facilite également l'utilisation de Docker, la gestion des services réseau et l'automatisation des tâches, tout en offrant de bonnes performances pour le déploiement et la gestion des conteneurs.



• *Docker :*

Docker a été utilisé afin de standardiser l'environnement de développement pour tous les membres de l'équipe. Grâce aux conteneurs, chaque développeur travaille dans le même environnement avec les mêmes dépendances et configurations, ce qui réduit considérablement les problèmes de compatibilité entre les machines.

Docker nous a également permis de simplifier le déploiement et les tests de l'application en regroupant tous les services nécessaires dans des conteneurs isolés et reproductibles. Enfin, le partage des images via Docker Hub a facilité la collaboration et assuré une cohérence entre les environnements de développement et de production.



• *Jira :*

Jira a été utilisé comme outil principal de gestion de projet et de suivi des tâches. Il nous a permis d'organiser les différentes étapes du projet, de répartir les responsabilités entre les membres de l'équipe et de suivre l'avancement du travail de manière claire et structurée.

Grâce à ses tableaux de bord et à son système de tickets, Jira a facilité la gestion collaborative du projet et amélioré la communication entre les membres. Son intégration avec GitHub nous a également permis de relier les commits et les pull



aux tâches correspondantes afin d'assurer un suivi plus professionnel du développement.

• *GitHub :*

GitHub a servi de plateforme de gestion et de versionnement du code source du projet. Grâce au système Git, il a permis de suivre les différentes modifications du code, de gérer les branches et de faciliter le travail collaboratif entre les membres de l'équipe.

Les fonctionnalités telles que les pull requests, les revues de code et GitHub Actions nous ont aidés à améliorer l'organisation du développement et à automatiser certaines tâches liées à l'intégration continue et au déploiement du projet.



• *Terraform :*

Terraform a été utilisé dans le cadre de l'automatisation de l'infrastructure et du déploiement de l'application. Cet outil d'Infrastructure as Code (IaC) nous a permis de créer et configurer automatiquement un serveur Linux destiné à héberger notre application.



Grâce à Terraform, l'infrastructure peut être reproduite facilement à partir de fichiers de configuration, ce qui garantit une meilleure cohérence entre les différents environnements. Son utilisation nous a également permis de découvrir les principes de l'automatisation et des pratiques DevOps modernes.

• *Laravel :*

Laravel est le framework PHP utilisé pour le développement du Back-end de notre application. Il nous a permis de créer une API sécurisée et bien structurée tout en facilitant la gestion des routes, des bases de données et de l'authentification.



Grâce à son architecture claire et à ses nombreux outils intégrés, Laravel a simplifié le développement des différentes fonctionnalités du projet ainsi que la communication entre le Front-end et la base de données.

- **Angular :**

Angular a été utilisé pour le développement de la partie Front-end de notre application web. Ce framework nous a permis de créer une interface utilisateur dynamique, moderne et réactive tout en facilitant l'organisation du code grâce à une architecture basée sur les composants.



Grâce à Angular, nous avons pu gérer efficacement la navigation entre les différentes pages de l'application, la communication avec l'API Laravel ainsi que l'affichage dynamique des données. Son intégration avec TypeScript nous a également aidés à maintenir un code plus structuré, réutilisable et facile à maintenir durant le développement du projet.

- **Nginx :**

Nginx a été utilisé comme serveur web et proxy au sein de notre environnement Docker. Il permet de gérer efficacement les requêtes entre les différents services de l'application et d'assurer la communication avec l'API Laravel.



Grâce à ses bonnes performances et à sa légèreté, Nginx contribue à améliorer la rapidité et la stabilité de l'application tout en facilitant la gestion des ports, de la sécurité et de la configuration réseau.

- **AWS :**

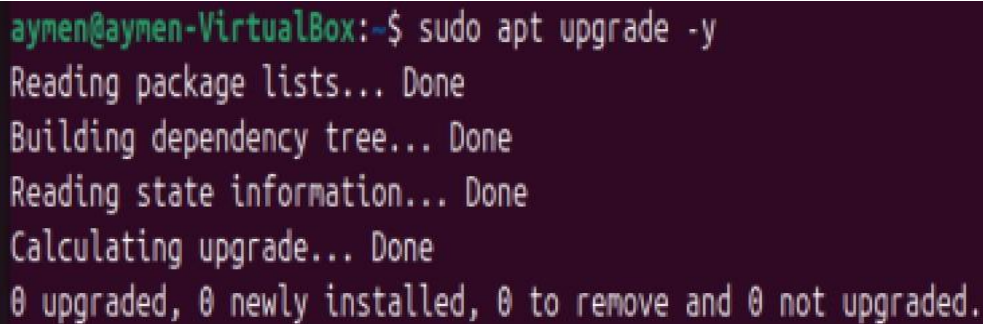
AWS (Amazon Web Services) est une plateforme de cloud computing qui permet de fournir des services d'hébergement, de stockage et de gestion d'infrastructure à distance. Dans notre projet, AWS a été utilisé pour héberger l'application web et le serveur Ubuntu Linux destiné au déploiement.



Grâce à son intégration avec Terraform, nous avons pu automatiser la création et la configuration de l'infrastructure, ce qui facilite le déploiement, améliore la fiabilité du système et garantit une infrastructure reproductible et facilement maintenable.

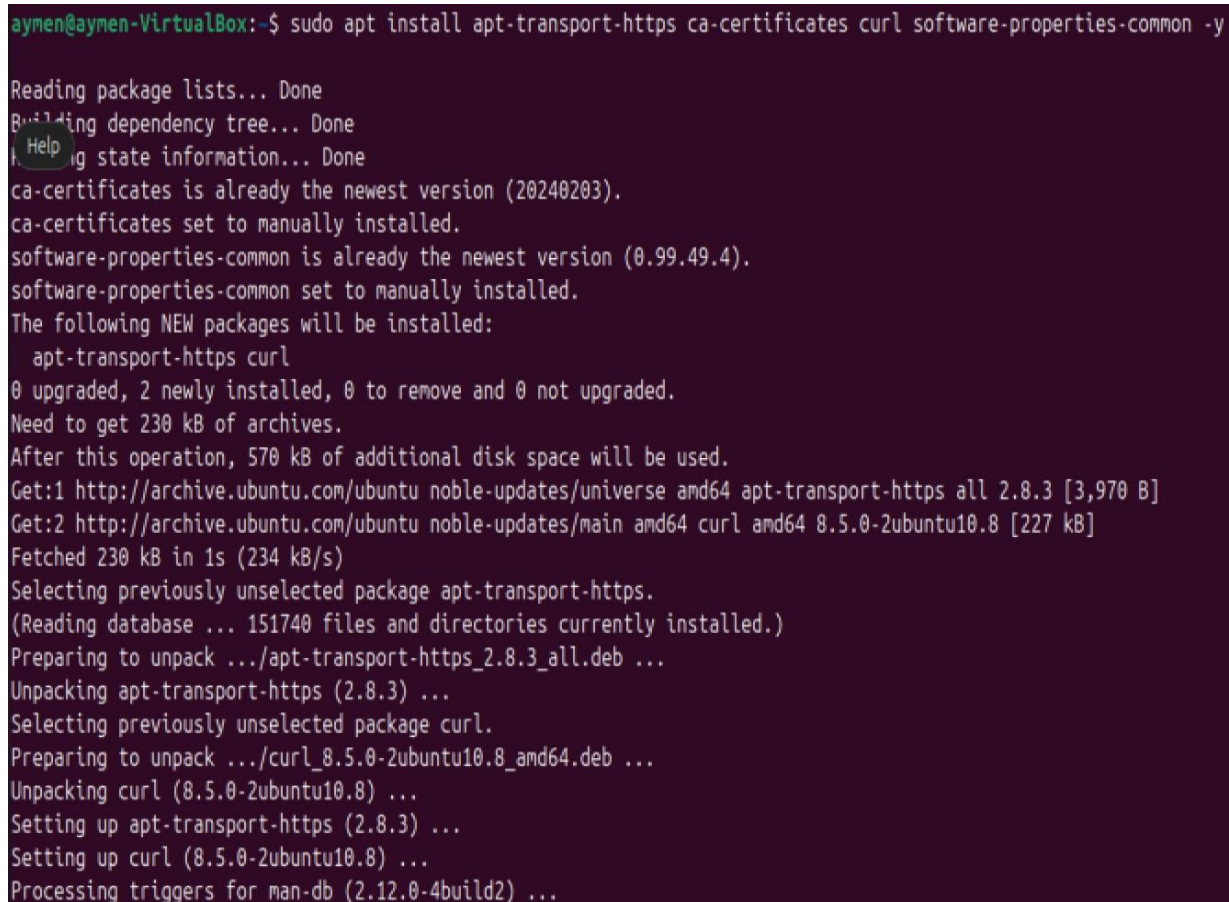
Chapitre 3 : Mise en place de l'environnement Docker

Installation de docker sur ubuntu 22.04:



```
aymen@aymen-VirtualBox:~$ sudo apt upgrade -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
```

Figure 3.1 – Commande d'installation de docker sur ubuntu



```
aymen@aymen-VirtualBox:~$ sudo apt install apt-transport-https ca-certificates curl software-properties-common -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ca-certificates is already the newest version (20240203).
ca-certificates set to manually installed.
software-properties-common is already the newest version (0.99.49.4).
software-properties-common set to manually installed.
The following NEW packages will be installed:
  apt-transport-https curl
0 upgraded, 2 newly installed, 0 to remove and 0 not upgraded.
Need to get 230 kB of archives.
After this operation, 570 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 apt-transport-https all 2.8.3 [3,970 B]
Get:2 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 curl amd64 8.5.0-2ubuntu10.8 [227 kB]
Fetched 230 kB in 1s (234 kB/s)
Selecting previously unselected package apt-transport-https.
(Reading database ... 151740 files and directories currently installed.)
Preparing to unpack .../apt-transport-https_2.8.3_all.deb ...
Unpacking apt-transport-https (2.8.3) ...
Selecting previously unselected package curl.
Preparing to unpack .../curl_8.5.0-2ubuntu10.8_amd64.deb ...
Unpacking curl (8.5.0-2ubuntu10.8) ...
Setting up apt-transport-https (2.8.3) ...
Setting up curl (8.5.0-2ubuntu10.8) ...
Processing triggers for man-db (2.12.0-4build2) ...
```

Figure 3.2 – Installation des paquets nécessaires via APT

```
aymen@aymen-VirtualBox:~$ echo "deb [arch=$(dpkg --print-architecture)]
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | su
do tee /etc/apt/sources.list.d/docker.list
[sudo] password for aymen:
deb [arch=amd64] https://download.docker.com/linux/ubuntu noble stable
```

Figure 3.3- Ajout du dépôt officiel Docker aux sources APT d'Ubuntu

```
aymen@aymen-VirtualBox:~$ sudo apt install docker-ce docker-ce-cli conta
inerd.io -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  docker-buildx-plugin docker-ce-rootless-extras docker-compose-plugin
  git git-man liberror-perl libslirp0 pigz slirp4netns
Suggested packages:
  cgroupfs-mount | cgroup-lite docker-model-plugin git-daemon-run
  | git-daemon-sysvinit git-doc git-email git-gui gitk gitweb git-cvs
  git-mediawiki git-svn
The following NEW packages will be installed:
  containerd.io docker-buildx-plugin docker-ce docker-ce-cli
  docker-ce-rootless-extras docker-compose-plugin git git-man
  liberror-perl libslirp0 pigz slirp4netns
0 upgraded, 12 newly installed, 0 to remove and 0 not upgraded.
```

Figure 3.4-Commande d'installation de Docker CE sur Ubuntu avec APT

```
aymen@aymen-VirtualBox:~$ sudo systemctl start docker
```

```
aymen@aymen-VirtualBox:~$ sudo docker run hello-world
```

```
Hello from Docker!
```

```
This message shows that your installation appears to be working correctly.
```

Figure 3.5-preuve de l'installation de docker

Configuration de dockerHub

Docker Hub a été utilisé comme plateforme de partage et de stockage des images Docker du projet. Il nous a permis de publier l'image Docker configurée pour notre environnement de développement afin que tous les membres de l'équipe puissent utiliser exactement la même configuration.

Grâce à Docker Hub, chaque membre pouvait facilement récupérer l'image Docker du projet sans avoir à reconfigurer manuellement l'environnement de développement. Cela a permis de garantir une meilleure cohérence entre les différentes machines et de faciliter la collaboration ainsi que le déploiement de l'application.

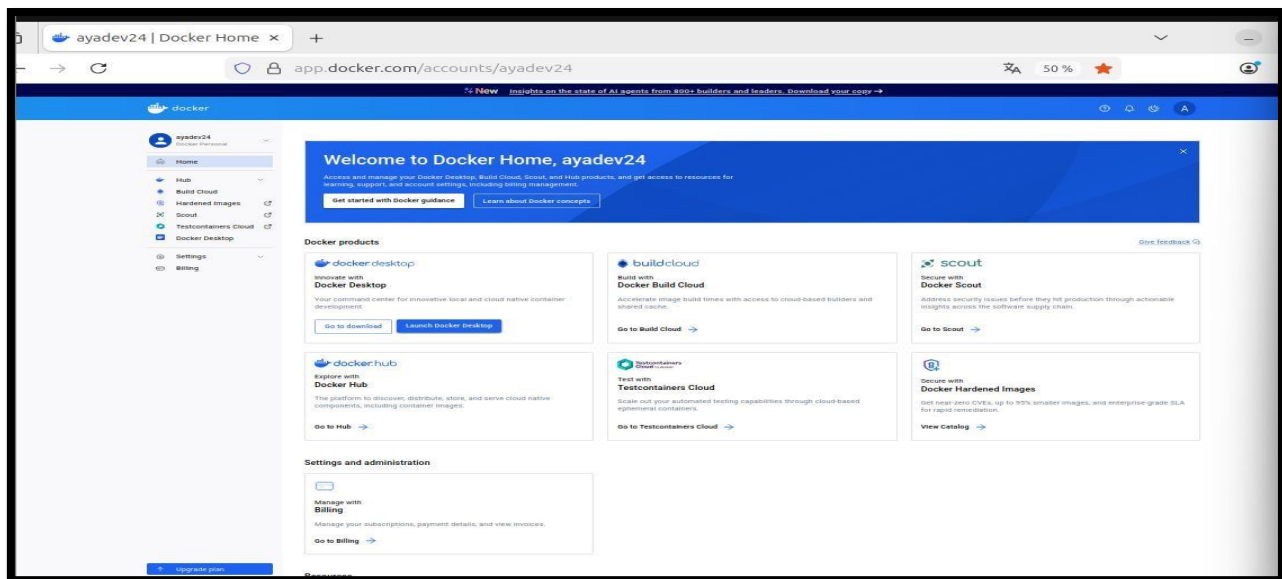


Figure 3.6-interface de dockerhub

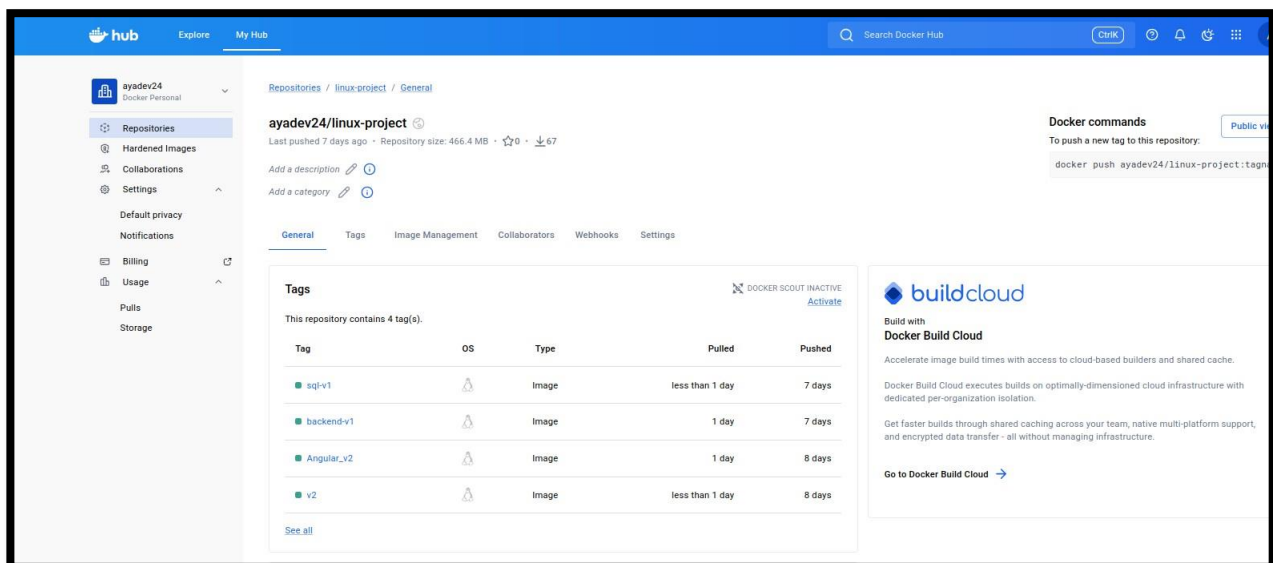


Figure 3.7-les images sur le repository dockerhub

Creation, build et publier les images dans Docker Hub :

- **Docker file pour mysql:**

Le fichier Dockerfile est utilisé pour créer une image Docker personnalisée destinée au service MySQL de notre projet. L'instruction `FROM mysql:8` permet d'utiliser l'image officielle de MySQL version 8 comme base de départ. Cette étape est importante car elle fournit directement un serveur MySQL prêt à être utilisé dans l'environnement Docker. L'instruction `WORKDIR /var/lib/mysql` définit le répertoire de travail principal du conteneur afin d'organiser correctement les fichiers liés à la base de données.

Les variables d'environnement `MYSQL_ROOT_PASSWORD`, `MYSQL_DATABASE`, `MYSQL_USER` et `MYSQL_PASSWORD` permettent de configurer automatiquement MySQL au démarrage du conteneur. Elles servent à créer la base de données `laravel_db`, un utilisateur dédié à l'application Laravel ainsi que son mot de passe, sans avoir besoin d'effectuer une configuration manuelle.

Les instructions `COPY my.cnf /etc/mysql/conf.d/my.cnf` et `COPY init.sql /docker-entrypoint-initdb.d/` ajoutent respectivement un fichier de configuration personnalisé et un script SQL dans le conteneur. Le fichier `my.cnf` permet de modifier certains paramètres de MySQL tandis que le fichier `init.sql` initialise automatiquement les tables et les données nécessaires au projet lors du premier lancement du conteneur. Enfin, l'instruction `EXPOSE 3306` permet d'ouvrir le port utilisé par MySQL afin d'assurer la communication avec Laravel et les autres services Docker.

- **Fichier init.sql :**

Le fichier init.sql est un script SQL exécuté automatiquement lors de l'initialisation de la base de données dans le conteneur Docker. Son rôle principal est de créer la structure complète de la base de données utilisée par l'application. Les premières instructions `CREATE DATABASE IF NOT EXISTS laravel_db` et `USE laravel_db` permettent de créer puis sélectionner la base de données principale du projet.

Le script crée ensuite plusieurs tables importantes comme `categories`, `users`, `livres` et `personal_access_tokens`. Chaque table contient différents champs adaptés aux besoins de l'application. Par exemple, la table `users` stocke les informations des utilisateurs comme le nom, l'email, le mot de passe et le rôle. La table `livres` contient les informations des livres tels que le titre, l'auteur, la description et la catégorie.

Le script utilise également des clés étrangères avec `FOREIGN KEY` afin de créer des relations entre certaines tables, notamment entre les livres et les catégories. Cela permet de garantir l'intégrité des données dans la base. Les instructions `INSERT INTO` ajoutent automatiquement certaines données initiales, comme des catégories prédéfinies et un compte administrateur, ce qui facilite les premiers tests et l'utilisation de l'application dès le démarrage du projet.

```

Code Blame 53 lines (47 loc) · 1.65 KB
1 CREATE DATABASE IF NOT EXISTS laravel_db;
2 USE laravel_db;
3
4 CREATE TABLE IF NOT EXISTS categories (
5     id INT AUTO_INCREMENT PRIMARY KEY,
6     nom VARCHAR(100) NOT NULL,
7     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
8 );
9
10 CREATE TABLE IF NOT EXISTS users (
11     id INT AUTO_INCREMENT PRIMARY KEY,
12     nom VARCHAR(255) NOT NULL,
13     prenom VARCHAR(255),
14     email VARCHAR(255) UNIQUE NOT NULL,
15     password VARCHAR(255) NOT NULL,
16     role ENUM('admin', 'user') DEFAULT 'user',
17     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
18 );
19
20 CREATE TABLE IF NOT EXISTS livres (
21     id INT AUTO_INCREMENT PRIMARY KEY,
22     titre VARCHAR(255) NOT NULL,
23     auteur VARCHAR(255) NOT NULL,
24     description TEXT,
25     image VARCHAR(500),
26     contenu VARCHAR(500),
27     category_id INT NOT NULL,
28     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
29     FOREIGN KEY (category_id) REFERENCES categories(id)
30 );
31
32 INSERT INTO categories (nom) VALUES
33 ('Informatique'), ('Sciences'), ('Littérature'), ('Histoire'), ('Economie');
34
35 INSERT INTO users (nom, prenom, email, password, role) VALUES
36 ('Admin', 'Super', 'admin@example.com', 'password', 'admin'),
37 ('admin', 'admin', 'admin@example.com', 'password', 'admin');
38
39
40 CREATE TABLE IF NOT EXISTS personal_access_tokens (
41     id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
42     tokenable_type VARCHAR(255) NOT NULL,
43     tokenable_id BIGINT UNSIGNED NOT NULL,
44     name VARCHAR(255) NOT NULL,
45     token VARCHAR(64) NOT NULL,
46     abilities TEXT,
47     last_used_at TIMESTAMP NULL,
48     expires_at TIMESTAMP NULL,
49     created_at TIMESTAMP NULL,
50     updated_at TIMESTAMP NULL,
51     UNIQUE KEY personal_access_tokens_token_unique (token),
52     INDEX personal_access_tokens_tokenable_type_tokenable_id_index (tokenable_type, tokenable_id)
53 );

```

Figure 3.8-Script SQL de création de la base de données

- **Fichier my.cnf :**

Le fichier my.cnf est un fichier de configuration personnalisé utilisé pour adapter le fonctionnement du serveur MySQL aux besoins du projet. La section [mysqld] indique que les paramètres définis concernent directement le serveur MySQL.

L'option sql_mode active un mode strict qui permet de limiter les erreurs liées aux données invalides et d'assurer une meilleure cohérence des informations stockées dans la base de données. Les paramètres character-set-server et collation-server configurent l'encodage UTF-8 afin de prendre en charge correctement les caractères spéciaux, les accents et les emojis dans l'application.

Le paramètre max_connections définit le nombre maximal de connexions simultanées autorisées sur le serveur MySQL afin d'améliorer la gestion des utilisateurs et d'éviter certains problèmes de surcharge. Même si certaines options sont facultatives, ce fichier permet globalement d'optimiser les performances, la stabilité et la compatibilité du serveur MySQL dans l'environnement Dockerisé du projet.

```

Code   Blame   14 lines (10 loc) - 329 Bytes
1      [mysqld]
2
3      #[mysqld]
4      sql_mode=STRICT_TRANS_TABLES,NO_ENGINE_SUBSTITUTION
5
6      # Encodage UTF-8 (très important)
7      character-set-server=utf8mb4
8      collation-server=utf8mb4_unicode_ci
9
10     # Performance basique
11     max_connections=100
12
13     # Désactiver certaines vérifications strictes (optionnel)
14     sql_mode=STRICT_TRANS_TABLES,NO_ENGINE_SUBSTITUTION

```

Figure 3.9-Extrait du fichier de configuration my.cnf définissant le mode SQL

- Build l'image de mysql :

```

fatima-zahra@fatima-zahra-VirtualBox:~/Documents/Projet-Linux/mysql$ docker build -t imagemysql .
[+] Building 572.9s (9/9) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 429B
=> [internal] load metadata for docker.io/library/mysql:8
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/4] FROM docker.io/library/mysql:8@sha256:43bd9764df60666fb2ba2cf8217dd17b3d2414c150005d2fdd07a0cd73d3b7
=> => resolve docker.io/library/mysql:8@sha256:43bd9764df60666fb2ba2cf8217dd17b3d2414c150005d2fdd07a0cd73d3b7
=> => sha256:182cd03161a6cc330cb3179e00ff03735060afe188a3c76b53de39ef68999458 5.33kB / 5.33kB
=> => sha256:6b97080db618a59c7c1070d873c7676532d8130e4bd4abdc2b6da3010cad2ef1 132.40MB / 132.40MB
=> => sha256:9b9c5f1fb9544e1b87160680d79634b2756df9965406bd997d83f86b1d3f189c 783.56kB / 783.56kB
=> => sha256:df58ba29e4716abfaa57ad92746073b6fdb2edc9ade56de9beacae40efc1ca54 334B / 334B
=> => sha256:97a156be315d3009791029be3f35d7721f77d2f89b4c7c375fc21c6df01f33f5 320B / 320B
=> => sha256:506bcb676d56150811d81dfeecf8c775ce1d8e9e3e42fcaff4cebedef91e668d 51.58MB / 51.58MB
=> => sha256:aa6e6468bcd71dd7e73c59c8ff211f3c3289b5689095efa58d80e7c7ccbdf766 2.60kB / 2.60kB
=> => sha256:8b565c834d59fd84b7e92785e9a9acfffe3a2cbdbc7ebfb500176b62589acb11 6.17MB / 6.17MB
=> => sha256:f035ba2a42b8a1b8f5674047539d50309c58d7c0bf025d946c8f93adad50a897 883B / 883B
=> => sha256:bb5107df7baaf29d90d3f3cd5c8b8d9bb0e9786e901a49be069e0d37e4c07bae 47.31MB / 47.31MB
=> => extracting sha256:bb5107df7baaf29d90d3f3cd5c8b8d9bb0e9786e901a49be069e0d37e4c07bae
=> => extracting sha256:f035ba2a42b8a1b8f5674047539d50309c58d7c0bf025d946c8f93adad50a897
=> => extracting sha256:9b9c5f1fb9544e1b87160680d79634b2756df9965406bd997d83f86b1d3f189c
=> => extracting sha256:8b565c834d59fd84b7e92785e9a9acfffe3a2cbdbc7ebfb500176b62589acb11
=> => extracting sha256:aa6e6468bcd71dd7e73c59c8ff211f3c3289b5689095efa58d80e7c7ccbdf766
=> => extracting sha256:df58ba29e4716abfaa57ad92746073b6fdb2edc9ade56de9beacae40efc1ca54
=> => extracting sha256:506bcb676d56150811d81dfeecf8c775ce1d8e9e3e42fcaff4cebedef91e668d
=> => extracting sha256:97a156be315d3009791029be3f35d7721f77d2f89b4c7c375fc21c6df01f33f5
=> => extracting sha256:6b97080db618a59c7c1070d873c7676532d8130e4bd4abdc2b6da3010cad2ef1
=> => extracting sha256:182cd03161a6cc330cb3179e00ff03735060afe188a3c76b53de39ef68999458
=> [internal] load build context

```

Figure 3-10-build de l'image mySql

```

fatima-zahra@fatima-zahra-VirtualBox:~/Documents/Projet-Linux/mysql$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
b69881d803e9   imagemysql    "docker-entrypoint.s..." About a minute ago Up About a minute 0.0.0.0:3306->3306/tcp, [
]:3306->3306/tcp, 33060/tcp
fatima-zahra@fatima-zahra-VirtualBox:~/Documents/Projet-Linux/mysql$

```

Figure 3.11-Affichage du conteneur MySQL actif via docker

- **Docker file pour Laravel :**

Ce fichier Dockerfile est utilisé pour créer l'image Docker du backend Laravel de notre projet. L'instruction FROM php:8.2-fpm utilise l'image officielle PHP 8.2 avec PHP-FPM comme base de départ. PHP-FPM permet d'exécuter efficacement les scripts PHP avec Nginx dans un environnement Docker.

Les instructions apt-get update et apt-get install permettent d'installer plusieurs dépendances nécessaires au fonctionnement de Laravel, comme Git, Curl, Zip ainsi que différentes bibliothèques utilisées pour gérer les images, les bases de données et certaines extensions PHP.

L'instruction docker-php-ext-install sert à installer plusieurs extensions PHP importantes comme pdo_mysql, mbstring, exif, pcntl, bcmath et pdo_sqlite. Ces extensions sont nécessaires pour assurer la compatibilité et le bon fonctionnement de Laravel avec MySQL et les différentes fonctionnalités du projet.

Le fichier installe également Composer grâce à l'instruction COPY --from=composer:latest, ce qui permet de gérer les dépendances PHP du projet Laravel. Ensuite, WORKDIR /var/www définit le dossier principal de travail du conteneur.

Les instructions COPY permettent de copier les fichiers du projet Laravel dans le conteneur Docker. La commande composer install --no-dev --optimize-autoloader installe les dépendances nécessaires au projet tout en optimisant les performances de chargement.

L'instruction RUN chmod -R 775 storage bootstrap/cache attribue les permissions nécessaires aux dossiers utilisés par Laravel pour le cache et le stockage des fichiers. Ensuite, un script entrypoint.sh est copié puis rendu exécutable afin d'automatiser certaines tâches au démarrage du conteneur.

Enfin, EXPOSE 9000 ouvre le port utilisé par PHP-FPM et l'instruction ENTRYPOINT permet de lancer automatiquement le script d'initialisation lorsque le conteneur démarre.

```

mon-projet-backend > Dockerfile
1 # Image PHP avec FPM
2 FROM php:8.2-fpm
3
4 # Installer dépendances système et les extensions php nécessaire pour Laravel
5 RUN apt-get update && apt-get install -y \
6     git \
7     curl \
8     zip \
9     unzip \
10    libpng-dev \
11    libonig-dev \
12    libxml2-dev \
13    libsqlite3-dev \
14    && docker-php-ext-install pdo pdo_mysql mbstring exif pcntl bcmath gd pdo_sqlite
15
16 # Installer Composer
17 COPY --from=composer:latest /usr/bin/composer /usr/bin/composer
18
19 # Définir dossier de travail
20 WORKDIR /var/www
21
22 # Copier tout le projet Laravel
23 COPY . /var/www/
24
25 # Installer dépendances PHP
26 RUN composer install --no-dev --optimize-autoloader
27
28 # Copier le reste du projet
29 COPY . .
30
31 # Permissions Laravel (build time - peut être écrasé par le volume)
32 RUN chmod -R 775 storage bootstrap/cache
33
34 # Copier et configurer le script d'entrée
35 COPY entrypoint.sh /usr/local/bin/entrypoint.sh
36 RUN chmod +x /usr/local/bin/entrypoint.sh
37
38 # Exposer port PHP-FPM
39 EXPOSE 9000
40
41 # Lancer PHP-FPM via entrypoint (reapplique les permissions au démarrage)
42 ENTRYPOINT ["/usr/local/bin/entrypoint.sh"]
43

```

Figure 3.12-Dockerfile pour l'environnement PHP-FPM dédié au projet Laravel

• Configuration Nginx :

Ce fichier correspond à la configuration du serveur Nginx utilisé dans notre environnement Docker afin de gérer les requêtes envoyées à l'application Laravel. L'instruction `listen 80` indique que Nginx écoute les requêtes HTTP sur le port 80, qui est le port standard des applications web. Les lignes `index index.php index.html` définissent les fichiers principaux qui seront chargés automatiquement lorsqu'un utilisateur accède au site.

L'instruction `root /var/www/public` définit le dossier public de Laravel comme répertoire principal du site web. Cela est important car Laravel utilise le dossier public pour exposer uniquement les fichiers accessibles par les utilisateurs et protéger les autres fichiers sensibles du projet.

Le bloc `location /` contient l'instruction `try_files`, qui permet à Nginx de vérifier si le fichier demandé existe réellement. Si ce n'est pas le cas, la requête est automatiquement redirigée vers `index.php`, ce qui permet à Laravel de gérer correctement le système de routage de l'application.

Le bloc `location ~ .php$` permet de traiter les fichiers PHP grâce à PHP-FPM. L'instruction `fastcgi_pass backend:9000` indique que les requêtes PHP sont envoyées vers le conteneur

backend fonctionnant sur le port 9000. Les paramètres `fastcgi_index`, `include fastcgi_params` et

`fastcgi_param SCRIPT_FILENAME` permettent de transmettre correctement les informations nécessaires à l'exécution des scripts PHP.

Enfin, le bloc `location ~ /\.ht` avec l'instruction `deny all` interdit l'accès aux fichiers cachés comme `.htaccess`, ce qui améliore la sécurité du serveur et protège certains fichiers sensibles.

```
1  server {
2      listen 80;
3      index index.php index.html;
4      root /var/www/public;
5
6      server_name localhost;
7
8      location / {
9          try_files $uri $uri/ /index.php?$query_string;
10     }
11
12     location ~ /\.php$ {
13         fastcgi_pass backend:9000;
14         fastcgi_index index.php;
15         include fastcgi_params;
16         fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
17     }
18
19     location ~ /\.ht {
20         deny all;
21     }
22 }
```

Figure 3.13-Configuration Nginx pour le serveur web du projet Laravel

- *Build l'image de Laravel:*

```

douae@douae-VirtualBox: ~/Projet-Linux/mon-projet-backend
conds 0.0.0.0:8000->9000/tcp, [::]:8000->9000/tcp mon_laravel
douae@douae-VirtualBox:~/Projet-Linux/mon-projet-backend$ docker login
Authenticating with existing credentials... [Username: douaeboufnar]

Info → To login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded
douae@douae-VirtualBox:~/Projet-Linux/mon-projet-backend$ docker tag imagelaravel douaeboufnar/imagelaravel:latest
douae@douae-VirtualBox:~/Projet-Linux/mon-projet-backend$ docker images

```

IMAGE	ID	DISK USAGE	CONTENT SIZE	In Use
douaeboufnar/imagelaravel:latest	d1e64cc2f53f	876MB	225MB	U
hello-world:latest	452a468a4bf9	25.9kB	9.49kB	U
imagelaravel:latest	d1e64cc2f53f	876MB	225MB	U

```

douae@douae-VirtualBox:~/Projet-Linux/mon-projet-backend$ docker push douaeboufnar/imagelaravel:latest
The push refers to repository [docker.io/douaeboufnar/imagelaravel]
f11fe9e4e821: Pushed
4f4fb700ef54: Pushed
21ade44bb56c: Pushed

```

Figure 3.14-Connexion à Docker Hub et push de l'image Laravel

```

douae@douae-VirtualBox: ~/Projet-Linux/mon-projet-backend
douae@douae-VirtualBox:~/Projet-Linux/mon-projet-backend$ docker push douaeboufnar/imagelaravel:latest
The push refers to repository [docker.io/douaeboufnar/imagelaravel]
f11fe9e4e821: Pushed
4f4fb700ef54: Pushed
21ade44bb56c: Pushed
105da8384709: Pushed
94b470014695: Pushed
07c7985f1f2f: Pushed
4d9f08ca4171: Pushed
18cad49ed3e0: Pushed
5435b2dcdf5c: Pushed
4f0f86da4d50: Pushed
d5af5dfa43d4: Pushed
e086798282ea: Pushed
e24121de25e6: Pushed
eacf10bf7fd6: Pushed
f08e1fed8626: Pushed
e715a3abea26: Pushed
0dbbd63ae63a: Pushed
28e2680bc4a9: Pushed
latest: digest: sha256:d1e64cc2f53fa7e663602235190ba97612a02bb2ee36beb8927e5035ada0b985 size: 856
douae@douae-VirtualBox:~/Projet-Linux/mon-projet-backend$

```

Figure 3.15- Push réussi de l'image Laravel vers Docker Hub

- **Docker file pour Angular :**

Ce fichier Dockerfile est utilisé pour créer l'image Docker du frontend Angular de notre projet. La première partie correspond à l'étape de build de l'application Angular. L'instruction `FROM node:20 AS build` utilise l'image officielle Node.js version 20 afin de disposer de l'environnement nécessaire pour compiler et construire le projet Angular.

L'instruction `WORKDIR /app` définit le dossier principal de travail à l'intérieur du conteneur. Ensuite, la commande `COPY package*.json ./` copie uniquement les fichiers liés aux dépendances du projet. Cette méthode permet d'optimiser le cache Docker et d'accélérer les reconstructions de l'image lorsque le code source change.

La commande `RUN npm install` installe toutes les dépendances nécessaires au fonctionnement de l'application Angular. Ensuite, l'instruction `COPY . .` copie l'ensemble des fichiers du projet dans le conteneur Docker.

La commande `RUN npm run build` lance la compilation du projet Angular en mode production afin de générer une version optimisée et prête pour le déploiement. Les fichiers générés sont placés dans le dossier `dist` du projet.

La deuxième partie du Dockerfile utilise `FROM nginx:alpine` afin d'utiliser Nginx comme serveur web léger pour héberger l'application Angular compilée. L'instruction `COPY -- from=build /app/dist/projetLinux/browser /usr/share/nginx/html/` copie les fichiers générés après le build Angular vers le dossier utilisé par Nginx pour servir les fichiers web.

L'instruction `COPY nginx.conf /etc/nginx/conf.d/default.conf` permet d'ajouter une configuration personnalisée de Nginx adaptée au projet. Ensuite, `EXPOSE 80` ouvre le port 80 afin de rendre l'application accessible depuis le navigateur.

Enfin, la commande `CMD ["nginx", "-g", "daemon off;"]` permet de lancer Nginx au démarrage du conteneur et de maintenir le serveur actif afin que l'application Angular reste accessible en permanence.

```

1 # Étape 1 : Build Angular
2 FROM node:20 AS build
3
4 WORKDIR /app
5
6 # Copier uniquement les fichiers de dépendances (optimisation cache)
7 COPY package*.json ./
8
9 # Installer les dépendances
10 RUN npm install
11
12 # Copier le reste du projet
13 COPY . .
14
15 # Build en production
16 RUN npm run build
17
18 # Étape 2 : Nginx (serveur web)
19 FROM nginx:alpine
20
21 # Copier les fichiers build Angular ( adapte le nom du projet)
22 COPY --from=build /app/dist/projetLinux/browser/ /usr/share/nginx/html/
23
24 # Copier la config Nginx
25 COPY nginx.conf /etc/nginx/conf.d/default.conf
26
27 # Exposer le port
28 EXPOSE 80
29
30 # Lancer Nginx
31 CMD ["nginx", "-g", "daemon off;"]
32

```

Figure 3-16-Configuration Docker

- **Build l'image de Angular :**

```

PS C:\Users\HP\Documents\Projet-Linux> docker build -t ayadev24/linux-project:Angular_v2 ./frontend
[+] Building 1.6s (16/16) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 330B
=> [internal] load metadata for docker.io/library/nginx:alpine
=> [internal] load metadata for docker.io/library/node:20
=> [auth] library/node:pull token for registry-1.docker.io
=> [auth] library/nginx:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [build 1/6] FROM docker.io/library/node:20@sha256:8789e1e0752d81088a085689c04fdb7a5b16e8102e353118a4b049bbf05db8ac
=> => resolve docker.io/library/node:20@sha256:8789e1e0752d81088a085689c04fdb7a5b16e8102e353118a4b049bbf05db8ac
=> [stage-1 1/2] FROM docker.io/library/nginx:alpine@sha256:5616878291a2eed594aee8db4dade5878cf7edcb475e59193904b198d9b830de
=> => resolve docker.io/library/nginx:alpine@sha256:5616878291a2eed594aee8db4dade5878cf7edcb475e59193904b198d9b830de
=> [internal] load build context
=> => transferring context: 1.49kB
=> CACHED [build 2/6] WORKDIR /app
=> CACHED [build 3/6] COPY package*.json ./
=> CACHED [build 4/6] RUN npm install
=> CACHED [build 5/6] COPY . .

```

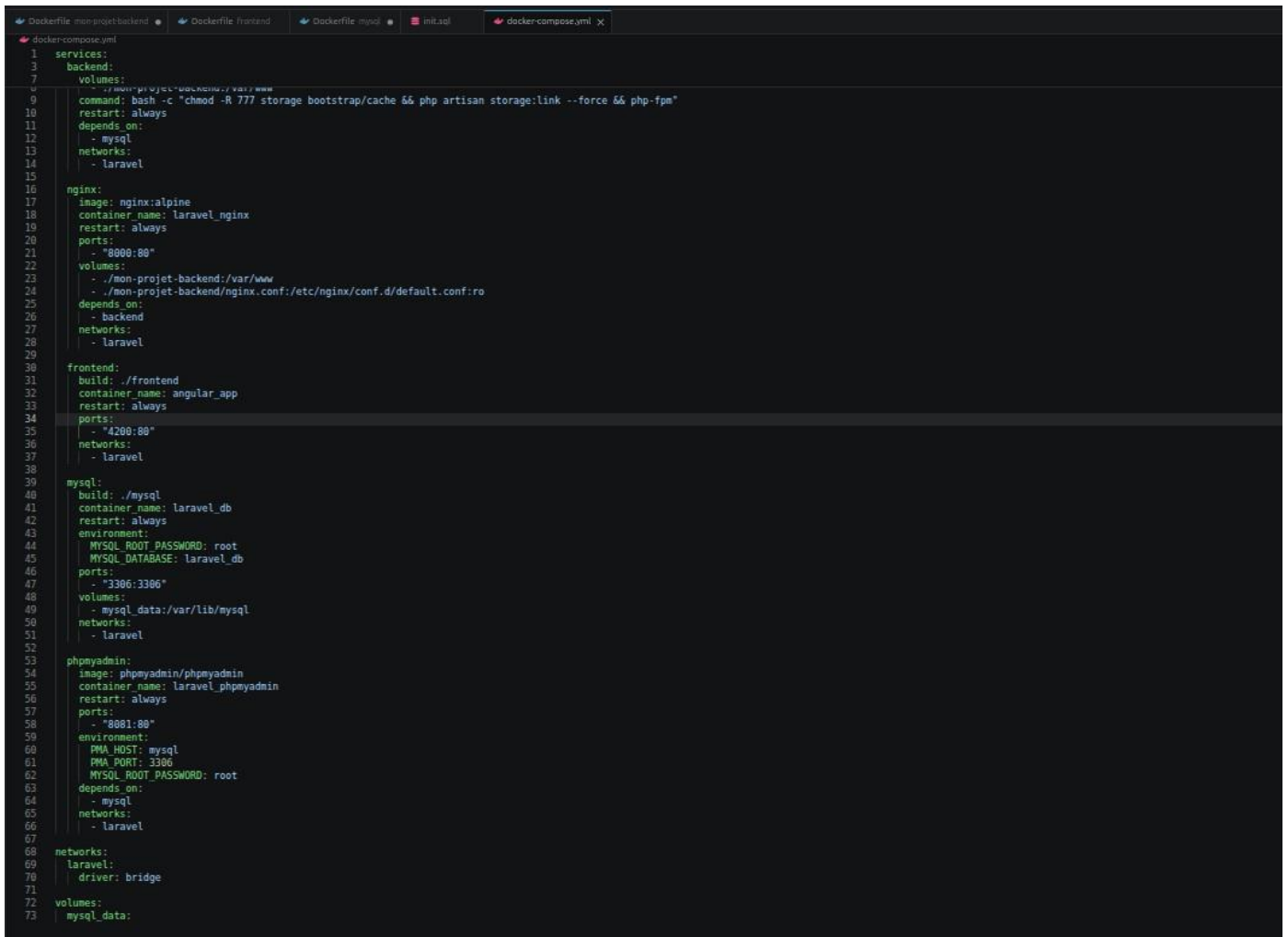
Figure 3.17- Build de l'image Docker Angular

```

PS C:\Users\HP\Documents\Projet-Linux> docker push ayadev24/linux-project:Angular_v2
The push refers to repository [docker.io/ayadev24/linux-project]
6a0ac1617861: Layer already exists
781ff50d2644: Layer already exists
4a8b0b2a5b19: Layer already exists
612c0c1df4c5: Layer already exists
aee4e54b3865: Layer already exists
453da7dbc73e: Layer already exists
fe0d20fa6ef2: Pushed
c9c46db05224: Layer already exists
583599bb7d38: Layer already exists
82736a35d0e7: Layer already exists
Angular_v2: digest: sha256:98f85fa16795c23dfaa9a31d4d558b946e8666ae4c7e4d97936595edffb0f8f0 size: 856
PS C:\Users\HP\Documents\Projet-Linux>

```

Docker compose :



```

1 services:
2   backend:
3     image: laravel/laravel
4     restart: always
5     depends_on:
6       - mysql
7     networks:
8       - laravel
9
10  nginx:
11    image: nginx:alpine
12    container_name: laravel_nginx
13    restart: always
14    ports:
15      - "8000:80"
16    volumes:
17      - ./non-projet-backend:/var/www
18      - ./non-projet-backend/nginx.conf:/etc/nginx/conf.d/default.conf:ro
19    depends_on:
20      - backend
21    networks:
22      - laravel
23
24  frontend:
25    build: ./frontend
26    container_name: angular_app
27    restart: always
28    ports:
29      - "4200:80"
30    networks:
31      - laravel
32
33  mysql:
34    build: ./mysql
35    container_name: laravel_db
36    restart: always
37    environment:
38      MYSQL_ROOT_PASSWORD: root
39      MYSQL_DATABASE: laravel_db
40    ports:
41      - "3306:3306"
42    volumes:
43      - mysql_data:/var/lib/mysql
44    networks:
45      - laravel
46
47  phpmyadmin:
48    image: phpmyadmin/phpmyadmin
49    container_name: laravel_phpmyadmin
50    restart: always
51    ports:
52      - "8081:80"
53    environment:
54      PMA_HOST: mysql
55      PMA_PORT: 3306
56      MYSQL_ROOT_PASSWORD: root
57    depends_on:
58      - mysql
59    networks:
60      - laravel
61
62  networks:
63    laravel:
64      driver: bridge
65
66  volumes:
67    mysql_data:

```

Figure 3.18-Orchestration des conteneurs pour le projet Linux

Structure d'un fichier docker-compose.yml :

La structure d'un fichier docker-compose.yml repose sur plusieurs sections essentielles permettant de définir et gérer l'ensemble des conteneurs d'une application Dockerisée.

▪ services (Définition des services) :

Cette section représente la partie principale du fichier docker-compose.yml. Elle permet de définir tous les conteneurs utilisés dans le projet, comme le backend Laravel, le frontend Angular, Nginx, MySQL et phpMyAdmin. Chaque service possède sa propre configuration afin d'assurer le bon fonctionnement de l'application.

• image :

Cette instruction permet de spécifier l'image Docker utilisée pour créer un conteneur. Elle peut

provenir de Docker Hub ou d'une image officielle déjà existante comme nginx:alpine ou phpmyadmin/phpmyadmin.

- container_name :

Cette instruction permet d'attribuer un nom personnalisé au conteneur afin de faciliter son identification et sa gestion dans Docker.

- ports :

Cette section permet de réaliser le mappage des ports entre la machine locale et le conteneur Docker. Elle rend les services accessibles depuis le navigateur ou depuis d'autres applications.

- volumes :

Les volumes permettent de partager des fichiers entre la machine hôte et les conteneurs Docker. Ils sont également utilisés pour conserver les données de manière persistante, même après l'arrêt ou la suppression des conteneurs.

- environment :

Cette section sert à définir des variables d'environnement utilisées à l'intérieur des conteneurs, comme les mots de passe, les noms des bases de données ou certaines configurations système.

- depends_on :

Cette instruction permet de définir les dépendances entre les services afin de garantir qu'un conteneur démarre uniquement après le lancement des services dont il dépend.

- restart :

La politique restart permet de définir le comportement du conteneur en cas d'arrêt ou d'erreur. Par exemple, l'option always redémarre automatiquement le conteneur.

- command :

Cette instruction permet d'exécuter une commande spécifique au démarrage du conteneur afin d'automatiser certaines tâches nécessaires au projet.

- networks :

Les réseaux Docker permettent aux différents conteneurs de communiquer entre eux dans un environnement isolé et sécurisé.

- volumes (Gestion du stockage persistant) :

Cette section permet de créer des volumes Docker utilisés pour sauvegarder les données importantes du projet, notamment les données de la base MySQL, afin qu'elles restent disponibles même après le redémarrage des conteneurs.

▪ networks (Gestion des réseaux Docker) :

Cette section permet de créer des réseaux personnalisés afin d'assurer la communication entre les différents services de l'application tout en garantissant une meilleure organisation et isolation du système Docker.

• **Notre docker compose :**

Le fichier docker-compose.yml est utilisé pour orchestrer et gérer l'ensemble des conteneurs Docker du projet. Il permet de lancer automatiquement tous les services nécessaires à l'application avec une seule commande, ce qui facilite énormément le développement et le déploiement. Dans ce projet, plusieurs services sont définis : backend Laravel, serveur Nginx, frontend Angular, base de données MySQL et phpMyAdmin.

La section services contient la définition de tous les conteneurs utilisés dans l'environnement Docker.

Le service backend :

correspond à l'application Laravel exécutée avec PHP-FPM. Les instructions volumes permettent de partager les fichiers du projet entre la machine locale et le conteneur Docker afin que les modifications soient directement prises en compte sans reconstruire l'image. La commande command exécute plusieurs opérations importantes au démarrage du conteneur, notamment la modification des permissions avec chmod, la création du lien de stockage Laravel avec php artisan storage:link et le lancement de PHP-FPM.

service nginx :

utilise l'image officielle nginx:alpine afin de servir l'application Laravel. Les ports 8000:80 permettent d'associer le port 80 du conteneur au port 8000 de la machine locale pour accéder à l'application depuis le navigateur. Les volumes servent à partager les fichiers Laravel ainsi que le fichier de configuration Nginx personnalisé. L'instruction depends_on garantit que le conteneur backend démarre avant Nginx afin d'assurer une bonne communication entre les services.

Le service frontend correspond à l'application Angular :

L'instruction build: ./frontend indique que l'image Docker doit être construite à partir du dossier frontend contenant le Dockerfile Angular. Le port 4200:80 permet d'accéder à l'interface Angular depuis le navigateur via le port 4200. L'option restart: always permet de redémarrer automatiquement le conteneur en cas d'arrêt ou d'erreur.

Le service mysql :

gère la base de données du projet. L'instruction build: ./mysql indique que l'image est construite à partir du Dockerfile MySQL personnalisé. Les variables d'environnement MYSQL_ROOT_PASSWORD et MYSQL_DATABASE servent à configurer automatiquement la base de données au démarrage du conteneur. Le port 3306:3306 permet à Laravel et aux autres services d'accéder à MySQL. La section volumes avec mysql_data:/var/lib/mysql permet de sauvegarder les données de la base même si le conteneur est supprimé ou redémarré.

Le service phpmyadmin :

utilise l'image officielle phpmyadmin/phpmyadmin afin de fournir une interface graphique pour gérer la base de données MySQL depuis le navigateur. Les variables PMA_HOST, PMA_PORT et MYSQL_ROOT_PASSWORD permettent de connecter automatiquement phpMyAdmin au conteneur MySQL. Le port 8081:80 rend l'interface accessible depuis le navigateur sur le port 8081.

La section networks :

crée un réseau Docker nommé laravel utilisant le driver bridge. Ce réseau permet à tous les conteneurs de communiquer entre eux de manière sécurisée et organisée. Enfin, la section volumes définit le volume mysql_data utilisé pour conserver les données MySQL de manière persistante même après l'arrêt des conteneurs.

Build du docker compose :

```

✓ Volume projet-linux mysql data Removed 0.0s
✓ Network projet-linux laravel Removed 0.1s
aya-dahbi@aya-dahbi-VirtualBox:~/Projet-Linux$ docker compose up --build
[+] Building 1.7s (47/47) FINISHED
=> [internal] load local bake definitions 0.0s
=> => reading from stdin 1.46kB 0.0s
=> [frontend internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 653B 0.0s
=> [backend internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 841B 0.0s
=> [mysql internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 429B 0.0s
=> [backend internal] load metadata for docker.io/library/composer:latest 0.8s
=> [backend internal] load metadata for docker.io/library/php:8.2-fpm 0.8s
=> [frontend internal] load metadata for docker.io/library/nginx:alpine 0.1s
=> [frontend internal] load metadata for docker.io/library/node:20 1.0s
=> [mysql internal] load metadata for docker.io/library/mysql:8 0.8s
=> [auth] library/php:pull token for registry-1.docker.io 0.0s
=> [auth] library/mysql:pull token for registry-1.docker.io 0.0s
=> [auth] library/node:pull token for registry-1.docker.io 0.0s
=> [auth] library/composer:pull token for registry-1.docker.io 0.0s
=> [mysql internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [backend internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [mysql 1/4] FROM docker.io/library/mysql:8@sha256:43bd9764df60666fb2ba2cf8217dd17b3d2414c1500 0.0s
=> => resolve docker.io/library/mysql:8@sha256:43bd9764df60666fb2ba2cf8217dd17b3d2414c150005d2fd 0.0s
=> [mysql internal] load build context 0.0s
=> => transferring context: 63B 0.0s
=> [backend stage-0 1/8] FROM docker.io/library/php:8.2-fpm@sha256:40c7b2994cae0d9d9c313513b2648 0.1s
=> => resolve docker.io/library/php:8.2-fpm@sha256:40c7b2994cae0d9d9c313513b26483d27eacaddf46fb9 0.0s
=> [backend internal] load build context 0.1s
=> => transferring context: 5.15kB 0.1s
=> [backend] FROM docker.io/library/composer:latest@sha256:dc292c5c0f95f526b051d4c341bf08e7e2b18 0.0s
=> => resolve docker.io/library/composer:latest@sha256:dc292c5c0f95f526b051d4c341bf08e7e2b18504c 0.0s
=> CACHED [mysql 2/4] WORKDIR /var/lib/mysql 0.0s
=> CACHED [mysql 3/4] COPY my.cnf /etc/mysql/conf.d/my.cnf 0.0s
=> [mysql 4/4] COPY init.sql /docker-entrypoint-initdb.d/ 0.1s
=> [mysql] exporting to image 0.3s

```

Figure 3.19-Lancement de l'application

```

gular_app | 2026/04/22 18:51:34 [notice] 1#1: start worker process 31
ravel_db | 2026-04-22 18:51:34+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.4.9-1.el9 started.
gular_app | 2026/04/22 18:51:34 [notice] 1#1: start worker process 32
ravel_db | 2026-04-22 18:51:35+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
ravel_phpmyadmin | AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 172.18.0.4. Set the 'ServerName' directive globally to suppress this message
ravel_db | 2026-04-22 18:51:35+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.4.9-1.el9 started.
ravel_phpmyadmin | AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 172.18.0.4. Set the 'ServerName' directive globally to suppress this message
ravel_phpmyadmin | [Wed Apr 22 18:51:35.176333 2026] [mpm_prefork:notice] [pid 1:tid 1] AH00163: Apache/2.4.65 (Debian) PHP/8.3.26 configured -- resuming normal operations
ravel_phpmyadmin | [Wed Apr 22 18:51:35.176681 2026] [core:notice] [pid 1:tid 1] AH00094: Command line: 'apache2 -D FOREGROUND'
ravel_nginx | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
ravel_nginx | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
ravel_nginx | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
ravel_nginx | 10-listen-on-ipv6-by-default.sh: info: can not modify /etc/nginx/conf.d/default.conf (read-only file system?)
ravel_nginx | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
ravel_nginx | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
ravel_nginx | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
ravel_nginx | /docker-entrypoint.sh: Configuration complete; ready for start up
ravel_nginx | 2026/04/22 18:51:35 [notice] 1#1: using the "epoll" event method
ravel_nginx | 2026/04/22 18:51:35 [notice] 1#1: nginx/1.29.8
ravel_nginx | 2026/04/22 18:51:35 [notice] 1#1: built by gcc 15.2.0 (Alpine 15.2.0)
ravel_nginx | 2026/04/22 18:51:35 [notice] 1#1: OS: Linux 6.17.0-20-generic
ravel_nginx | 2026/04/22 18:51:35 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
ravel_nginx | 2026/04/22 18:51:35 [notice] 1#1: start worker processes
ravel_nginx | 2026/04/22 18:51:35 [notice] 1#1: start worker process 20
ravel_nginx | 2026/04/22 18:51:35 [notice] 1#1: start worker process 21
ravel_nginx | 2026/04/22 18:51:35 [notice] 1#1: start worker process 22
ravel_nginx | 2026/04/22 18:51:35 [notice] 1#1: start worker process 23
ravel_db | 2026-04-22 18:51:35+00:00 [Note] [Entrypoint]: Initializing database files
ravel_db | 2026-04-22T18:51:35.490067Z 0 [System] [MY-015017] [Server] MySQL Server Initialization - start.
ravel_db | 2026-04-22T18:51:35.492964Z 0 [Warning] [MY-010915] [Server] 'NO_ZERO_DATE', 'NO_ZERO_IN_DATE' and 'ERROR_FOR_DIVISION_BY_ZERO' sql modes should be used with strict mode. They will be merged with strict mode in a future release.
ravel_db | 2026-04-22T18:51:35.493049Z 0 [System] [MY-013169] [Server] /usr/sbin/mysqld (mysqld 8.4.9) initializing of server in progress as process 79
ravel_db | 2026-04-22T18:51:35.499924Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
ravel_db | 2026-04-22T18:51:35.739644Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.

```

Figure 3.20-Sortie des logs lors du lancement de l'application

```

>=> exporting manifest sha256:d832355af468b2ea80a19abbb8ecb1becelb17e6adfbf3491e4b1ee78ccb492a 0.0s
>=> exporting config sha256:3070d147559e98b9d96870744f7056ff6fdddc1a3352689baaf99be5e66a8e4 0.0s
>=> exporting attestation manifest sha256:96150107a52f49f4991dd03d990d213b0032f46238eb61be8af8 0.0s
>=> exporting manifest list sha256:d527f4257e7b650d1b466b537fe5413b53ef60a1b089e232b14473fcl9a9 0.0s
>=> naming to docker.io/library/projet-linux-frontend:latest 0.0s
>=> unpacking to docker.io/library/projet-linux-frontend:latest 0.0s
> [mysql] resolving provenance for metadata file 0.0s
> [frontend] resolving provenance for metadata file 0.0s
> [backend] resolving provenance for metadata file 0.0s
] up 10/10
Image projet-linux-backend Built 1.8s
Image projet-linux-frontend Built 1.8s
Image projet-linux-mysql Built 1.8s
Network projet-linux laravel Created 0.1s
Volume projet-linux_mysql_data Created 0.0s
Container angular_app Created 0.1s
Container laravel_db Created 0.1s
Container laravel_app Created 0.1s
Container laravel_phpmyadmin Created 0.1s
Container laravel_nginx Created 0.1s
Attaching to angular_app, laravel_app, laravel_db, laravel_nginx, laravel_phpmyadmin
angular_app | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
angular_app | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
angular_app | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
angular_app | 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
angular_app | 10-listen-on-ipv6-by-default.sh: info: /etc/nginx/conf.d/default.conf differs from the packaged version
angular_app | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
angular_app | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
angular_app | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
angular_app | /docker-entrypoint.sh: Configuration complete; ready for start up
angular_app | 2026/04/22 18:51:34 [notice] 1#1: using the "epoll" event method
angular_app | 2026/04/22 18:51:34 [notice] 1#1: nginx/1.29.8
angular_app | 2026/04/22 18:51:34 [notice] 1#1: built by gcc 15.2.0 (Alpine 15.2.0)
angular_app | 2026/04/22 18:51:34 [notice] 1#1: OS: Linux 6.17.0-20-generic
angular_app | 2026/04/22 18:51:34 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
angular_app | 2026/04/22 18:51:34 [notice] 1#1: start worker processes
angular_app | 2026/04/22 18:51:34 [notice] 1#1: start worker process 29
angular_app | 2026/04/22 18:51:34 [notice] 1#1: start worker process 30
angular_app | 2026/04/22 18:51:34 [notice] 1#1: start worker process 31

```

Figure 3.21- Résultat de la construction

```

• aya-dahbi@aya-dahbi-VirtualBox:~/Projet-Linux$ docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
angular_app         projet-linux-frontend "/docker-entrypoint..." frontend    29 minutes ago Up 21 minutes 0.0.0.0:4200->80/tcp
laravel_app         projet-linux-backend "docker-php-entrypoi..." backend    29 minutes ago Up 21 minutes 9000/tcp
laravel_db          projet-linux-mysql  "docker-entrypoint.s..." mysql      29 minutes ago Up 23 minutes 0.0.0.0:3306->3306/tcp
laravel_nginx       nginx:alpine        "/docker-entrypoint..." nginx      29 minutes ago Up 21 minutes 0.0.0.0:8000->80/tcp
laravel_phpmyadmin  phpmyadmin/phpmyadmin "/docker-entrypoint..." phpmyadmin 29 minutes ago Up 21 minutes 0.0.0.0:8081->80/tcp

```

Figure 3.22- État des conteneurs Docker après déploiement

```

aya-dahbi@aya-dahbi-VirtualBox:~/Projet-Linux$ docker compose up -d
WARN[0000] Found orphan containers ([projet-linux-backend-run-3d4594d503a2 projet-linux-backend-run-7d5d1073218c]) for
removed or renamed this service in your compose file, you can run this command with the --remove-orphans flag to clean
[+] up 5/5
✓ Container laravel_db Running
✓ Container angular_app Running
✓ Container laravel_app Running
✓ Container laravel_phpmyadmin Running
✓ Container laravel_nginx Running
aya-dahbi@aya-dahbi-VirtualBox:~/Projet-Linux$

```

Figure 3.23- Démarrage des conteneurs en arrière-plan

Chapitre 4 : Utilisation de CI et tests de validation

Dans le cadre de ce projet, nous avons mis en place un pipeline d'intégration continue en utilisant **GitHub Actions**, un service d'automatisation intégré nativement à GitHub

But d'utilisation de GitHub :

GitHub a été utilisé comme plateforme principale de gestion du code source et de collaboration entre les membres de l'équipe. Chaque développeur travaillait sur une branche dédiée à sa fonctionnalité afin de développer et tester ses modifications de manière indépendante avant de les fusionner dans la branche principale main. GitHub a également permis de centraliser le projet, de suivre l'historique des modifications et de faciliter la gestion des versions grâce au système Git. Les fonctionnalités de *pull requests*, de gestion des branches et de résolution des conflits ont amélioré la collaboration et assuré une meilleure organisation du travail en équipe.

But d'utilisation de GitHub Actions :

GitHub Actions a été choisi pour automatiser les processus d'intégration continue et de déploiement continu (CI/CD) directement depuis GitHub. Cette solution permet d'exécuter automatiquement les tests Laravel et Angular, de construire les images Docker et de déployer l'application sur le serveur AWS après chaque mise à jour du projet. GitHub Actions a été préféré à des outils comme Jenkins car il est directement intégré à GitHub, plus simple à configurer et ne nécessite pas de serveur supplémentaire pour fonctionner. Son intégration native avec les repositories GitHub facilite la gestion des workflows CI/CD et améliore l'automatisation du projet.

Configuration des secrets GitHub :

Avant de mettre en place les pipelines CI/CD, nous avons configuré les secrets du dépôt GitHub. Ces secrets permettent de stocker les informations sensibles de manière sécurisée, sans les exposer dans le code source.

Les six secrets configurés sont les suivants :

- **AWS_ACCESS_KEY_ID** : clé d'accès AWS permettant au pipeline de s'authentifier auprès des services Amazon Web Services.
- **AWS_SECRET_ACCESS_KEY** : clé secrète associée à la clé d'accès AWS, utilisée conjointement pour sécuriser la connexion à l'infrastructure cloud.
- **DOCKERHUB_TOKEN** : token d'authentification Docker Hub permettant au pipeline de publier les images Docker sur le registre.

- **DOCKERHUB_USERNAME** : nom d'utilisateur du compte Docker Hub utilisé pour se connecter au registre lors du push des images.
- **SERVER_IP** : adresse IP du serveur de production AWS sur lequel l'application est déployée via SSH.
- **SSH_PRIVATE_KEY** : clé privée SSH permettant au pipeline de se connecter au serveur de production de manière sécurisée sans mot de passe

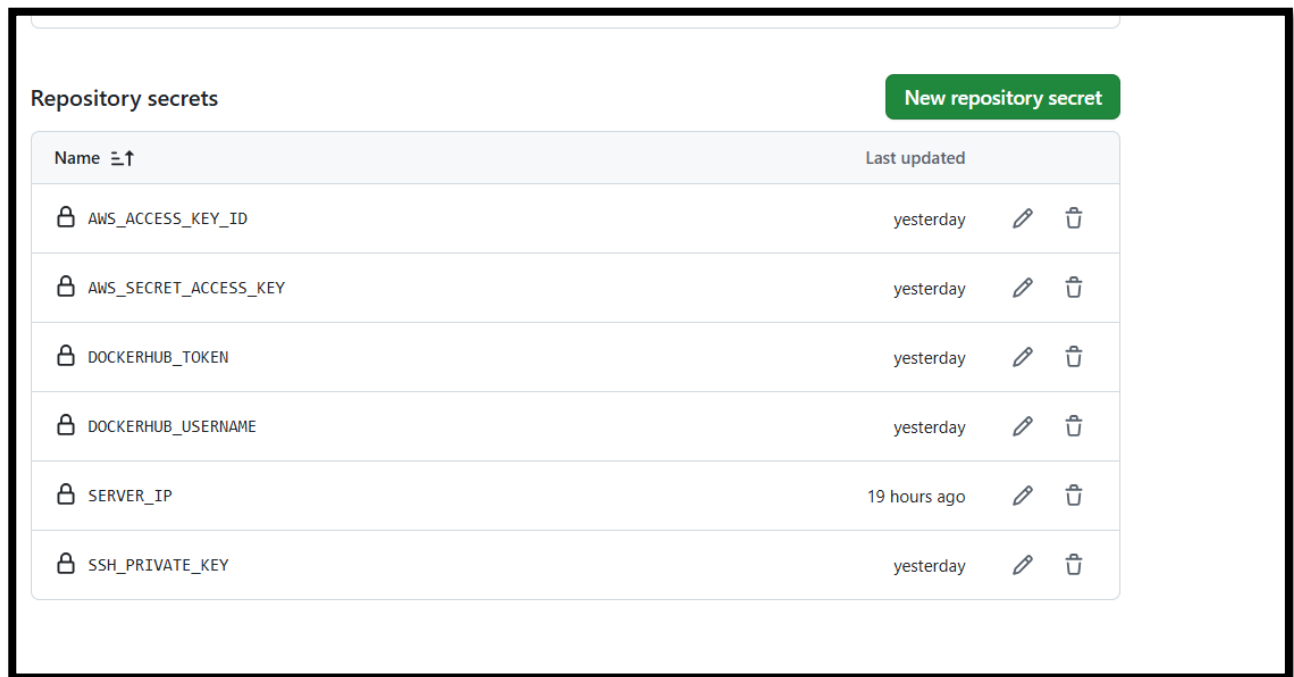


Figure 4.1- les secrets de github

Définition et rôle :

Le workflow mis en place est composé de deux jobs qui s'exécutent en parallèle sur des environnements Ubuntu isolés et propres, fournis par GitHub.

Le premier job prend en charge la partie backend. Il installe PHP 8.2, récupère les dépendances via Composer, configure l'environnement à partir du fichier .env.example, génère une clé applicative, puis exécute l'ensemble de la suite de tests Laravel via la commande php artisan test.

Le second job prend en charge la partie frontend. Il installe Node.js 20, récupère les dépendances npm, puis lance les tests Angular via npm test.

Fichier de configuration :

Le pipeline est entièrement décrit dans un fichier YAML placé à l'emplacement .github/workflows/ci.yml à la racine du dépôt. Ce fichier est versionné avec le reste du code source, ce qui signifie que toute modification du pipeline est traçable et réversible au même titre que le code applicatif.

```

Projet-Linux / .github / workflows / ci.yml
Code Blame 47 lines (44 loc) · 1.33 KB
3   on:
4     push:
5       branches: ['**']
6     pull_request:
7       branches: ['**']
8
9   jobs:
10    test-laravel:
11      name: Tests Laravel (Backend)
12      runs-on: ubuntu-latest
13      steps:
14        - name: Checkout du code
15          uses: actions/checkout@v4
16        - name: Installer PHP 8.2
17          uses: shivammathur/setup-php@v2
18          with:
19            php-version: '8.2'
20        - name: Installer les dépendances
21          working-directory: ./mon-projet-backend
22          run: composer install --no-interaction --prefer-dist
23        - name: Configurer l'environnement
24          working-directory: ./mon-projet-backend
25          run: |
26            cp .env.example .env
27            php artisan key:generate
28        - name: Lancer les tests
29          working-directory: ./mon-projet-backend
30          run: php artisan test
31
32    test-angular:
33      name: Tests Angular (Frontend)

```

```

31
32    test-angular:
33      name: Tests Angular (Frontend)
34      runs-on: ubuntu-latest
35      steps:
36        - name: Checkout du code
37          uses: actions/checkout@v4
38        - name: Installer Node.js 20
39          uses: actions/setup-node@v4
40          with:
41            node-version: '20'
42        - name: Installer les dépendances
43          working-directory: ./frontend
44          run: npm install
45        - name: Lancer les tests
46          working-directory: ./frontend
47          run: npm test -- --watch=false --browsers=ChromeHeadless || echo "Tests skipped or failed"

```

Figure 4.2- Workflow CI/CD GitHub Actions pour les tests Laravel (backend) et Angular -frontend

Demonstration :

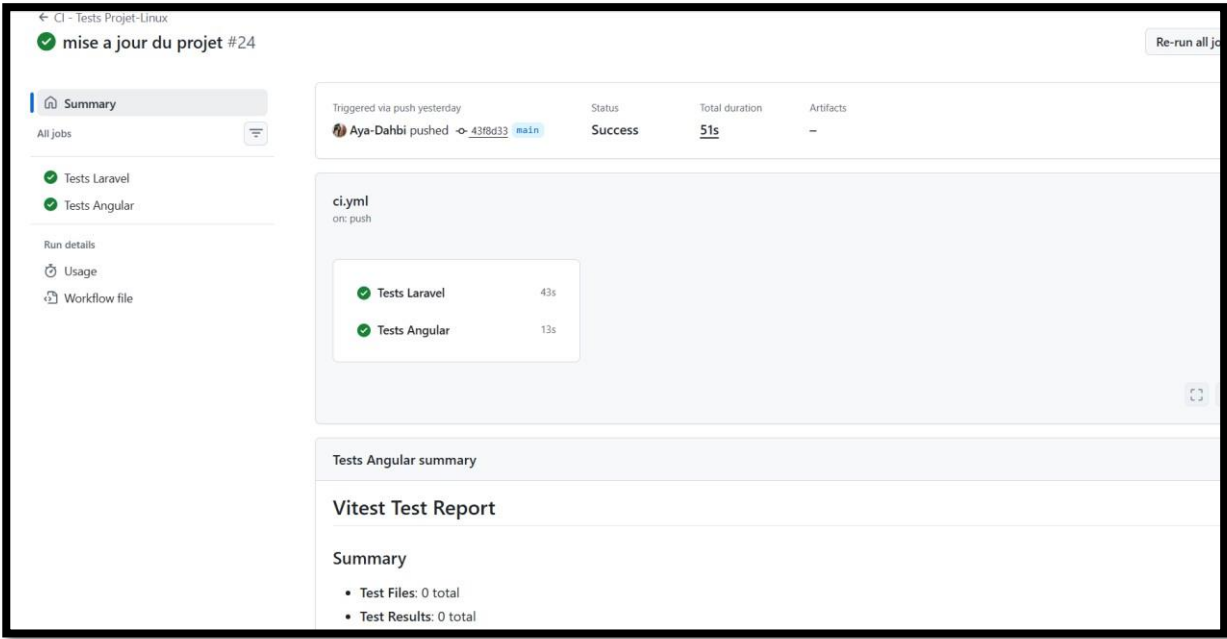


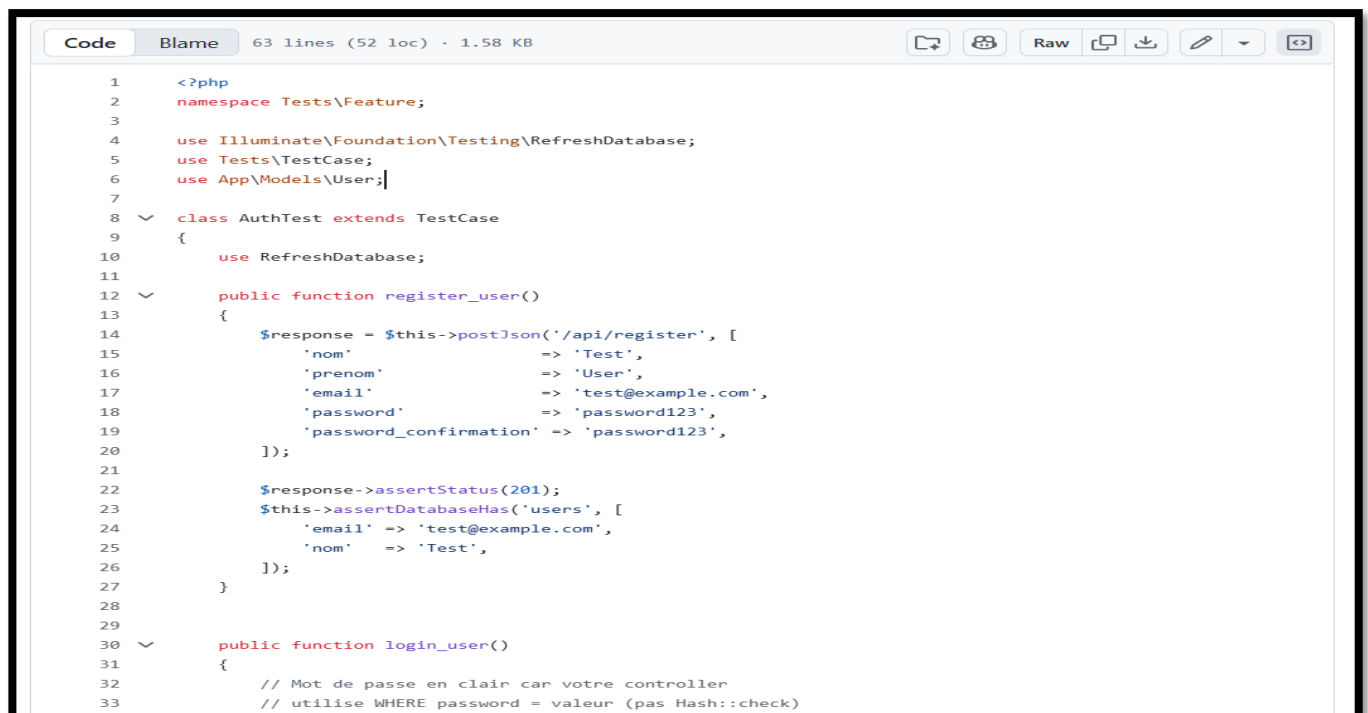
Figure 4.3- Résultat du pipeline CI GitHub Actions

 after trauma CI - Tests (Laravel & Angular) #59: Commit a48d538 pushed by reda465	test	 May 16, 4:55 PM GMT+1  51s ...
 fixed db CI - Tests (Laravel & Angular) #58: Commit 14fc4d2 pushed by reda465	test	 May 16, 4:11 PM GMT+1  42s ...
 corriger test livre CI - Tests (Laravel & Angular) #57: Commit b0753c7 pushed by douae-boufnar	pipeline-douae	 May 16, 4:00 PM GMT+1  53s ...
 test de livres CI - Tests (Laravel & Angular) #56: Commit 51fee77 pushed by douae-boufnar	pipeline-douae	 May 16, 3:55 PM GMT+1  48s ...
 update de pipeline CI - Tests (Laravel & Angular) #55: Commit fce9aa2 pushed by douae-boufnar	pipeline-douae	 May 16, 3:43 PM GMT+1  44s ...
 fixing error in phpMyAdmin interface CI - Tests (Laravel & Angular) #54: Commit 2c43dc1 pushed by reda465	test	 May 16, 3:34 PM GMT+1  45s ...
 Mettre à jour .gitignore et retirer les fichiers suivis CI - Tests (Laravel & Angular) #53: Commit 189b1f8 pushed by douae-boufnar	pipeline-douae	 May 16, 3:31 PM GMT+1  44s ...
 expose 8081 de ma machine CI - Tests (Laravel & Angular) #52: Commit b1e30fc pushed by reda465	test	 May 16, 3:24 PM GMT+1  46s ...
 Mettre à jour .gitignore et retirer les fichiers suivis CI - Tests (Laravel & Angular) #51: Commit 189b1f8 pushed by douae-boufnar	douae	 May 16, 3:22 PM GMT+1  51s ...

Figure 4.4- Suivi des commits et des résultats d'intégration continue

les tests d'authentification côté backend. :

Les tests d'authentification backend consistent à vérifier automatiquement que les endpoints de l'API liés à l'inscription et à la connexion des utilisateurs fonctionnent correctement. Ils simulent des requêtes HTTP réelles envoyées à l'API et contrôlent que les réponses retournées correspondent au comportement attendu, aussi bien dans les cas de succès que dans les cas d'échec.



```
1  <?php
2  namespace Tests\Feature;
3
4  use Illuminate\Foundation\Testing\RefreshDatabase;
5  use Tests\TestCase;
6  use App\Models\User;
7
8  class AuthTest extends TestCase
9  {
10     use RefreshDatabase;
11
12     public function register_user()
13     {
14         $response = $this->postJson('/api/register', [
15             'nom' => 'Test',
16             'prenom' => 'User',
17             'email' => 'test@example.com',
18             'password' => 'password123',
19             'password_confirmation' => 'password123',
20         ]);
21
22         $response->assertStatus(201);
23         $this->assertDatabaseHas('users', [
24             'email' => 'test@example.com',
25             'nom' => 'Test',
26         ]);
27     }
28
29     public function login_user()
30     {
31         // Mot de passe en clair car votre controller
32         // utilise WHERE password = valeur (pas Hash::check)
```

Figure 4.5- Tests d'authentification Lara

-register user

Teste l'inscription d'un nouvel utilisateur. Il envoie une requête POST à /api/register avec les données nécessaires et vérifie que le serveur répond avec le code 201 et que l'utilisateur est bien enregistré en base de données.

-login user

Teste la connexion avec des identifiants valides. Il crée d'abord un utilisateur en base, puis envoie une requête POST à /api/login et vérifie que le serveur répond avec le code 200 et retourne bien les informations de l'utilisateur.

-login fail

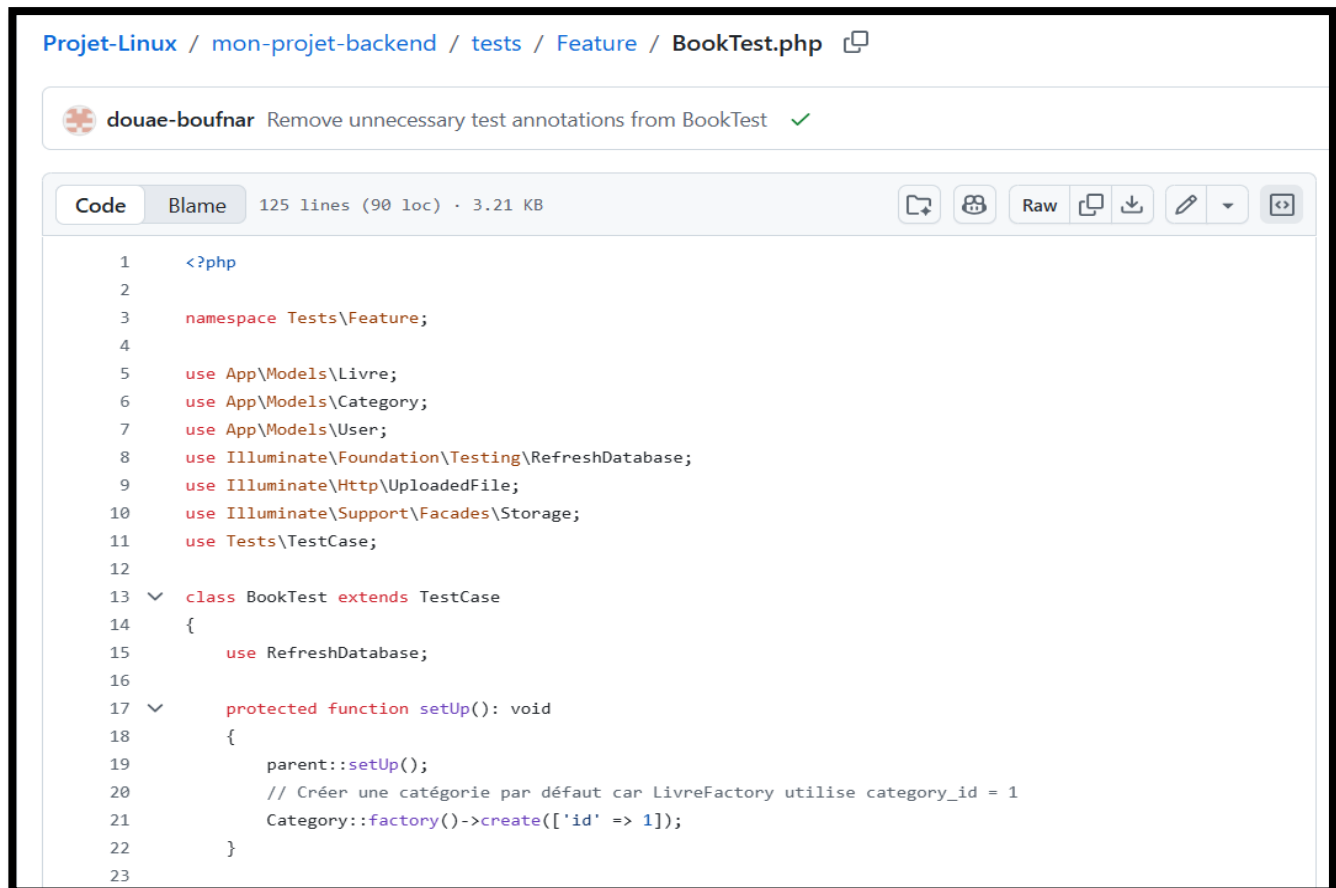
Teste le rejet d'une connexion avec des identifiants incorrects. Il envoie une requête POST à /api/login avec un email et un mot de passe inexistant et vérifie que le serveur répond avec le code 401, confirmant que l'accès est bien refusé.

-Tests des livres : bokkTest.php :

Ces tests couvrent l'ensemble des opérations de la gestion des livres, c'est-à-dire la création, la lecture, la modification et la suppression. Ils garantissent que l'API des livres fonctionne correctement dans tous les cas possibles, aussi bien pour les actions de base que pour les accès protégés.

- Structure générale :

Avant chaque test, une catégorie par défaut est créée en base de données avec l'identifiant 1, car la factory des livres en a besoin pour fonctionner correctement.



```
Projet-Linux / mon-projet-backend / tests / Feature / BookTest.php

douae-boufnar Remove unnecessary test annotations from BookTest ✓

Code Blame 125 lines (90 loc) · 3.21 KB

1 <?php
2
3 namespace Tests\Feature;
4
5 use App\Models\Livre;
6 use App\Models\Category;
7 use App\Models\User;
8 use Illuminate\Foundation\Testing\RefreshDatabase;
9 use Illuminate\Http\UploadedFile;
10 use Illuminate\Support\Facades\Storage;
11 use Tests\TestCase;
12
13 class BookTest extends TestCase
14 {
15     use RefreshDatabase;
16
17     protected function setUp(): void
18     {
19         parent::setUp();
20         // Créer une catégorie par défaut car LivreFactory utilise category_id = 1
21         Category::factory()->create(['id' => 1]);
22     }
23 }
```

Figure 4.6-Classe BookTest héritant de TestCase

it can list all books

Teste la récupération de la liste des livres. Il crée 3 livres en base via la factory, envoie une requête GET à /api/books et vérifie que le serveur répond avec le code 200 et retourne bien 3 livres dans la réponse.

```

23
24
25 ✓ public function it_can_list_all_books()
26 {
27     Livre::factory()->count(3)->create();
28
29     $response = $this->getJson('/api/books');
30
31     $response->assertStatus(200)
32         ->assertJsonCount(3);
33 }
34

```

- it can create a book

Teste l'ajout d'un nouveau livre. Il simule un stockage de fichiers avec `Storage::fake`, envoie une requête POST à `/api/books` avec les données du livre ainsi qu'une image et un PDF fictifs, et vérifie que le serveur répond avec le code 201 et que le livre est bien enregistré en base de données.

```

46
47 ✓ public function it_can_create_a_book()
48 {
49     Storage::fake('public');
50
51     $data = [
52         'titre' => 'Nouveau Livre',
53         'auteur' => 'Auteur Test',
54         'description' => 'Une description de test',
55         'category_id' => 1,
56         'image' => UploadedFile::fake()->image('cover.jpg'),
57         'contenu' => UploadedFile::fake()->create('book.pdf', 100)
58     ];
59
60     $response = $this->postJson('/api/books', $data);
61
62     $response->assertStatus(201)
63         ->assertJsonPath('livre.titre', 'Nouveau Livre');
64
65     $this->assertDatabaseHas('livres', [
66         'titre' => 'Nouveau Livre',
67         'auteur' => 'Auteur Test'
68     ]);
69 }
70

```

Figure 4.7--Test fonctionnel de l'API POST `/api/books`

- it can delete a book

Teste la suppression d'un livre. Il crée un livre en base, envoie une requête DELETE à `/api/books/{id}` et vérifie que le serveur répond avec le code 200 et que le livre a bien disparu de la base de données via `assertDatabaseMissing`

```

92
93
94  ✓ public function it_can_delete_a_book()
95      {
96          $book = Livre::factory()->create();
97
98          $response = $this->deleteJson("/api/books/{ $book->id}");
99
100         $response->assertStatus(200);
101         $this->assertDatabaseMissing('livres', ['id' => $book->id]);
102     }
103

```

Figure 4.8- Test de suppression d'un livre

- guest cannot access protected read route

Teste qu'un utilisateur non connecté ne peut pas accéder au contenu d'un livre. Il envoie une requête GET à `/api/books/{id}/read` sans authentification et vérifie que le serveur répond avec le code 401, confirmant que la route est bien protégée.

```

103
104  ✓ public function guest_cannot_access_protected_read_route()
105      {
106          $book = Livre::factory()->create();
107
108          $response = $this->getJson("/api/books/{ $book->id}/read");
109
110         $response->assertStatus(401);
111     }
112

```

Z

Figure 4.9- Test de sécurité Laravel accès interdit aux invités sur la route protégée

- authenticated user can access protected read route

Teste qu'un utilisateur connecté peut accéder au contenu d'un livre. Il crée un utilisateur et un livre, puis envoie une requête GET à `/api/books/{id}/read` en étant authentifié via Sanctum et vérifie que le serveur répond avec le code 200 et retourne bien l'URL du contenu du livre

```
113
114 ✓ public function authenticated_user_can_access_protected_read_route()
115 {
116     $user = User::factory()->create();
117     $book = Livre::factory()->create(['contenu' => 'http://example.com/book.pdf']);
118
119     $response = $this->actingAs($user, 'sanctum')
120                 ->getJson("/api/books/{ $book->id }/read");
121
122     $response->assertStatus(200)
123                 ->assertJsonPath('url', 'http://example.com/book.pdf');
124 }
125 }
```

Figure 4-13- Test d'accès non autorisé à une route protégée

Les tests mis en place couvrent les deux fonctionnalités principales de l'application : l'authentification et la gestion des livres.

Du côté de l'authentification, trois tests vérifient que l'inscription d'un nouvel utilisateur, la connexion avec des identifiants valides et le rejet d'une connexion incorrecte fonctionnent correctement.

Chapitre 5 : Utilisation de Terraform :

Dans le cadre de ce projet, Terraform a été utilisé pour automatiser la création et la configuration de l'infrastructure cloud nécessaire au déploiement de l'application. Terraform est un outil d'Infrastructure as Code (IaC) développé par HashiCorp qui permet de définir, provisionner et gérer des ressources cloud à travers des fichiers de configuration déclaratifs.

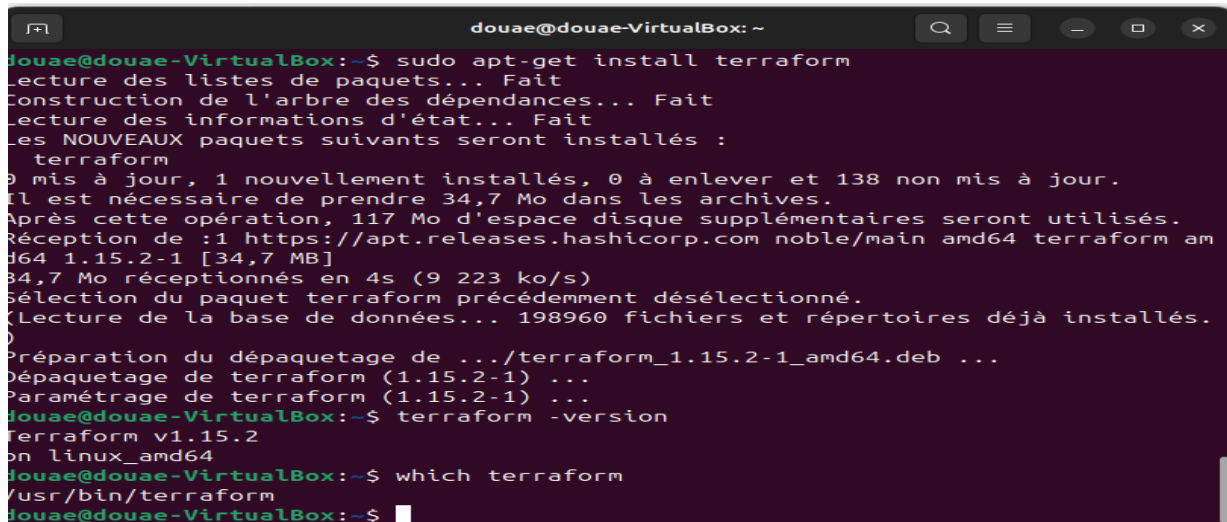
Grâce à Terraform, il est possible de décrire l'état souhaité de l'infrastructure (serveur EC2, groupes de sécurité, réseau, etc.) dans des fichiers .tf, puis de laisser Terraform calculer automatiquement les actions nécessaires pour atteindre cet état. Cette approche garantit une infrastructure reproductible, versionnée et facile à maintenir.

-L'installation de Terraform :

L'installation de Terraform sur Ubuntu s'effectue via le gestionnaire de paquets apt. La commande suivante permet d'installer Terraform directement depuis les dépôts officiels HashiCorp :

Après l'installation, il est possible de vérifier que Terraform est bien installé et d'afficher la version installée avec la commande `terraform --version`.

Capture d'écran — Installation de Terraform :



```
douae@douae-VirtualBox: ~
douae@douae-VirtualBox:~$ sudo apt-get install terraform
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Les NOUVEAUX paquets suivants seront installés :
  terraform
0 mis à jour, 1 nouvellement installés, 0 à enlever et 138 non mis à jour.
Il est nécessaire de prendre 34,7 Mo dans les archives.
Après cette opération, 117 Mo d'espace disque supplémentaires seront utilisés.
Réception de :1 https://apt.releases.hashicorp.com noble/main amd64 terraform am
d64 1.15.2-1 [34,7 MB]
34,7 Mo réceptionnés en 4s (9 223 ko/s)
Sélection du paquet terraform précédemment désélectionné.
(Lecture de la base de données... 198960 fichiers et répertoires déjà installés.
)
Préparation du dépaquetage de .../terraform_1.15.2-1_amd64.deb ...
Dépaquetage de terraform (1.15.2-1) ...
Paramétrage de terraform (1.15.2-1) ...
douae@douae-VirtualBox:~$ terraform -version
Terraform v1.15.2
on linux_amd64
douae@douae-VirtualBox:~$ which terraform
/usr/bin/terraform
douae@douae-VirtualBox:~$
```

Figure 5.1- installation de terraform

-Structure du projet Terraform :

-Le dossier terraform du projet contient quatre fichiers de configuration principaux, chacun ayant un rôle bien défini dans la gestion de l'infrastructure :

main.tf	Fichier principal : définit les ressources AWS (provider, instance EC2, script user_data)
variables.tf	Déclare les variables utilisées (région, AMI, type d'instance, clé SSH, URL GitHub...)
outputs.tf	Définit les sorties après apply (ID instance, IP publique, commande SSH, URL application)
security.tf	Crée le Security Group AWS avec les règles d'accès entrantes/sortantes (SSH, HTTP)

-Fichier main.tf

Le fichier main.tf constitue le cœur de la configuration Terraform. Il définit le provider AWS à utiliser ainsi que la ressource principale : l'instance EC2 qui hébergera l'application.

-Configuration du Provider AWS :

Le bloc terraform déclare la version du provider AWS requise (~> 5.0), ce qui garantit la compatibilité tout en autorisant les mises à jour mineures. Le provider aws est ensuite configuré avec la région définie dans les variables.

```
resource "aws_instance" "projet_linux_server" {
  ami                = var.aws_ami_id
  instance_type      = var.instance_type
  key_name           = var.key_name
  associate_public_ip_address=true
  vpc_security_group_ids = [aws_security_group.projet_linux_sg.id]
  user_data_replace_on_change = true

  # Stockage 20 GB gp2 (Free Tier)
  root_block_device {
    volume_size = 20
    volume_type = "gp2"
  }
}

terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}
```

Figure 5.2- définition du fournisseur AWS requis

Ressource EC2 : aws_instance :

-La ressource `aws_instance` crée une instance EC2 Ubuntu sur AWS. Elle utilise l'AMI Ubuntu spécifiée dans les variables, le type d'instance `t3.micro` (éligible au Free Tier), la clé SSH `terraform-ec2` pour l'accès sécurisé, et associe automatiquement une adresse IP publique.

Un disque racine de 20 Go de type `gp2` est configuré. Le bloc `user_data` contient un script Bash qui s'exécute automatiquement au premier démarrage de l'instance pour :

- Mettre à jour les paquets et installer Git
- Ajouter un espace SWAP de 2 Go pour éviter les crashes mémoire lors du build Angular (Exit 137)
- Installer Docker via Snap
- Cloner le dépôt GitHub sur la branche test
- Lancer l'application avec `docker compose up -d`

```
resource "aws_instance" "projet_linux_server" {
  ami            = var.aws_ami_id
  instance_type  = var.instance_type
  key_name       = var.key_name
  associate_public_ip_address=true
  vpc_security_group_ids = [aws_security_group.projet_linux_sg.id]
  user_data_replace_on_change = true

  # Stockage 20 GB gp2 (Free Tier)
  root_block_device {
    volume_size = 20
    volume_type = "gp2"
  }
}
```

```
resource "aws_instance" "projet_linux_server" {
  # Script d'installation automatique via Snap (Idéal pour Ubuntu très récent)
  user_data = <<-EOF
    #!/bin/bash
    # 1. Mise à jour
    apt-get update -y
    apt-get install -y git

    # 2. Ajout de la mémoire SWAP (2Go) pour éviter le crash OOM (Exit 137) d'Angular
    fallocate -l 2G /swapfile
    chmod 600 /swapfile
    mkswap /swapfile
    swapon /swapfile
    echo '/swapfile none swap sw 0 0' >> /etc/fstab

    # 3. Installation de Docker via Snap
    snap install docker

    # On attend que Snap finisse de bien configurer Docker
    sleep 10

    # 4. Récupération du code
    cd /home/ubuntu
    git clone -b main ${var.github_repo_url} projet
    cd projet

    # 5. Lancement de l'application
    sudo docker compose up -d

    # Fix permissions
    chown -R ubuntu:ubuntu /home/ubuntu/projet
  EOF
}
```

Figure 5.3- Script d'automatisation Terraform pour déployer l'application

-Fichier variables.tf :

Le fichier variables.tf centralise toutes les valeurs configurables du projet. Cela facilite la réutilisation et la modification de l'infrastructure sans toucher au code principal. Les variables définies sont les suivantes :

aws_region	eu-west-3 (Paris)
instance_type	t3.micro (Free Tier)
aws_ami_id	ami-0be40a46b4111e7f5 (Ubuntu sur eu-west-3)
key_name	terraform-ec2 (nom de la Key Pair SSH sur AWS)
project_name	projet-linux
github_repo_url	https://github.com/douae-boufnar/Projet-Linux

```

terraform > variables.tf > variable "aws_region"
1  variable "aws_region" {
2    | default = "eu-west-3"
3  | }
4
5  variable "instance_type" {
6    | description = "Type d'instance EC2 (Free Tier)"
7    | default     = "t3.micro"
8  | }
9
10
11 variable "aws_ami_id" {
12 | description="c'est l'id de notre image ubuntu"
13 | default="ami-0be40a46b4111e7f5"
14 | }
15
16 variable "key_name" {
17 | description = "Nom de la Key Pair SSH"
18 | default     = "terraform-ec2"
19 | }
20
21 variable "project_name" {
22 | description = "Nom du projet"
23 | default     = "projet-linux"
24 | }
25
26 variable "github_repo_url" {
27 | description = "URL du dépôt GitHub du projet"
28 | default     = "https://github.com/douae-boufnar/Projet-Linux"
29 | }

```

```

terraform > variables.tf > variable "aws_region"
1  variable "aws_region" {
2      default = "eu-west-3"
3  }
4
5  variable "instance_type" {
6      description = "Type d'instance EC2 (Free Tier)"
7      default     = "t3.micro"
8  }
9
10 variable "aws_ami_id" {
11     description="c'est l'id de notre image ubuntu"
12     default="ami-0be40a46b4111e7f5"
13 }
14
15 variable "key_name" {
16     description = "Nom de la Key Pair SSH"
17     default     = "terraform-ec2"
18 }
19
20 variable "project_name" {
21     description = "Nom du projet"
22     default     = "projet-linux"
23 }
24
25 variable "github_repo_url" {
26     description = "URL du dépôt GitHub du projet"
27     default     = "https://github.com/douae-boufnar/Projet-Linux"
28 }
29

```

Figure 5.4- Définition des variables Terraform pour le déploiement AWS EC2

Le fichier security.tf- Security Group :

Le fichier security.tf définit le Security Group AWS qui contrôle les flux réseau entrants et sortants de l'instance EC2. Deux règles d'entrée (ingress) sont configurées :

Port 22 (SSH) — ouvert depuis n'importe quelle adresse IP (0.0.0.0/0) pour l'accès administrateur

Port 80 (HTTP) — ouvert depuis n'importe quelle adresse IP pour le trafic web via Nginx

Une règle egress autorise tout le trafic sortant (protocole -1, tous les ports). Le Security Group est tagué avec le nom du projet pour une identification facile dans la console AWS.

```

resource "aws_security_group" "projet_linux_sg" {
    name      = "${var.project_name}-sg"
    description = "Security Group pour Projet Linux (Reverse Proxy Nginx)"

    # SSH - accès admin
    ingress {
        description = "SSH"
        from_port   = 22
        to_port     = 22
        protocol    = "tcp"
        cidr_blocks = ["0.0.0.0/0"]
    }
}

```

```
# HTTP - Trafic Web Principal (Nginx Reverse Proxy)
ingress {
  description = "HTTP (Nginx Reverse Proxy)"
  from_port   = 80
  to_port     = 80
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
}

# Tout le trafic sortant autorisé
egress {
  from_port   = 0
  to_port     = 0
  protocol    = "-1"
  cidr_blocks = ["0.0.0.0/0"]
}

tags = {
  Name = "${var.project_name}-sg"
}
```

Figure 5.5-- Définition des règles de pare-feu AWS pour l'instance EC2 du projet Linux

Fichier out.tf :

Le fichier outputs.tf définit les valeurs affichées après l'exécution de terraform apply. Ces outputs permettent de récupérer facilement les informations essentielles de l'instance déployée :

instance_id	L'identifiant unique de l'instance EC2 créée sur AWS
public_ip	L'adresse IP publique assignée à l'instance
ssh_command	La commande SSH complète pour se connecter au serveur
app_url	L'URL publique de l'application via Nginx

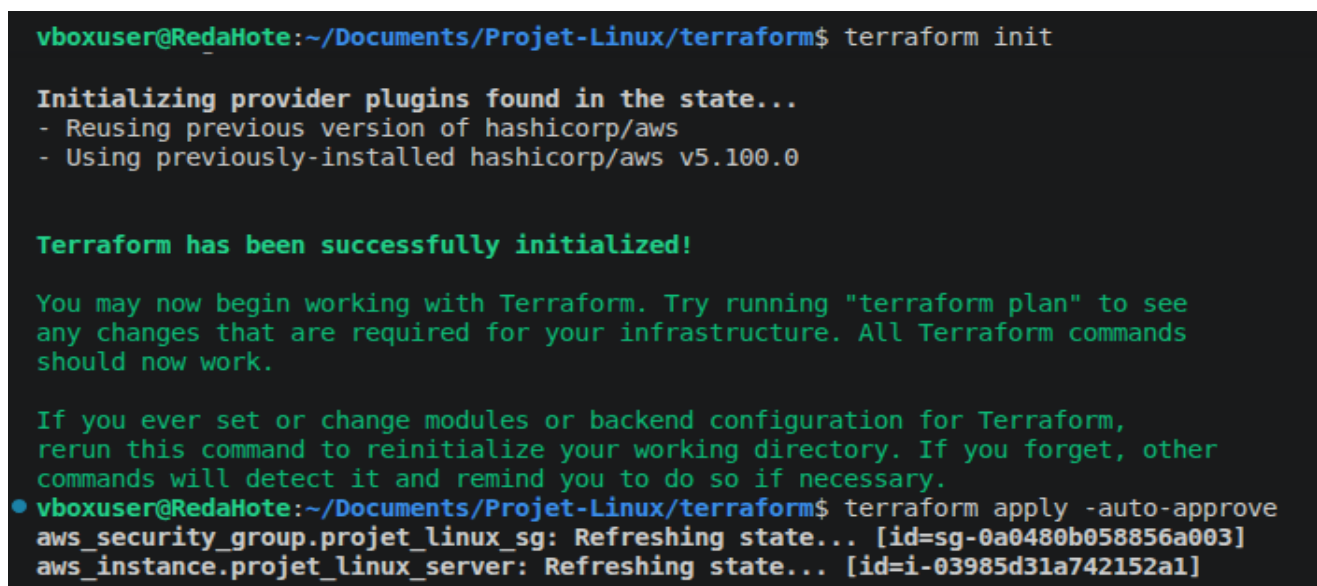
```
terraform > variables.tf outputs.tf x main.tf {} terraform.tfstate M terraform.tfstate.backup M security.tf
1 output "instance_id" {
2   description = "ID de l'instance EC2"
3   value       = aws_instance.projet_linux_server.id
4 }
5
6
7
8 output "public_ip" {
9   description = "Adresse IP publique de l'instance EC2"
10  value       = aws_instance.projet_linux_server.public_ip
11 }
12
13 output "ssh_command" {
14   description = "Commande SSH pour se connecter au serveur"
15   value       = "ssh -i ~/.ssh/terraform-ec2.pem ubuntu@${aws_instance.projet_linux_server.public_ip}"
16 }
17
18 output "app_url" {
19   description = "URL publique de l'application (Nginx Reverse Proxy)"
20   value       = "http://${aws_instance.projet_linux_server.public_ip}"
21 }
22
```

Deploiement infrastructure :

-Le déploiement de l'infrastructure Terraform se déroule en trois étapes principales : l'initialisation, la planification et l'application. Les commandes suivantes sont exécutées depuis le répertoire terraform/ du projet.

Initialisation –terraform init

-La commande terraform init est la première commande à exécuter dans un nouveau dossier Terraform. Elle initialise le répertoire de travail en téléchargeant les plugins des providers déclarés (ici le provider AWS) et en configurant le backend.



```
vboxuser@RedaHote:~/Documents/Projet-Linux/terraform$ terraform init

Initializing provider plugins found in the state...
- Reusing previous version of hashicorp/aws
- Using previously-installed hashicorp/aws v5.100.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
• vboxuser@RedaHote:~/Documents/Projet-Linux/terraform$ terraform apply -auto-approve
aws_security_group.projeto_linux_sg: Refreshing state... [id=sg-0a0480b058856a003]
aws_instance.projeto_linux_server: Refreshing state... [id=i-03985d31a742152a1]
```

Figure 5.6- Sortie terminal montrant la réutilisation du provider AWS

Application – terraform apply -auto-approve :

-La commande terraform apply -auto-approve applique le plan d'exécution et crée l'infrastructure sur AWS sans demander de confirmation manuelle. Terraform crée toutes les ressources déclarées dans les fichiers .tf : le Security Group et l'instance EC2.

Une fois l'application terminée, Terraform affiche un résumé des ressources créées ainsi que les valeurs des outputs définis dans outputs.tf : l'ID de l'instance, l'adresse IP publique, la commande SSH et l'URL de l'application.

L'état de l'infrastructure est sauvegardé dans le fichier terraform.tfstate, qui sert de source de vérité pour Terraform lors des futures opérations.

```
vboxuser@RedaHote:~/Documents/Projet-Linux/terraform$ terraform apply -auto-approve
aws_instance.projet_linux_server: Still destroying... [id=i-03985d31a742152a1, 00m10s elapsed]
aws_instance.projet_linux_server: Still destroying... [id=i-03985d31a742152a1, 00m20s elapsed]
aws_instance.projet_linux_server: Still destroying... [id=i-03985d31a742152a1, 00m30s elapsed]
aws_instance.projet_linux_server: Still destroying... [id=i-03985d31a742152a1, 00m40s elapsed]
aws_instance.projet_linux_server: Destruction complete after 41s
aws_instance.projet_linux_server: Creating...
aws_instance.projet_linux_server: Still creating... [00m10s elapsed]
aws_instance.projet_linux_server: Creation complete after 14s [id=i-04b76ffa87dc8d595]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

Outputs:
app_url = "http://35.180.24.77"
instance_id = "i-04b76ffa87dc8d595"
public_ip = "35.180.24.77"
ssh_command = "ssh -i ~/.ssh/terraform-ec2.pem ubuntu@35.180.24.77"
```

Figure 5.7-Destruction et recréation de l'instance EC2 avec Terraform

Connexion SSH a l'instance EC2

Après le déploiement, l'instance EC2 est accessible via SSH en utilisant la clé privée terraform-ec2.pem générée lors de la création de la Key Pair sur AWS. L'adresse IP publique est récupérée depuis la sortie Terraform (output public_ip).

Lors de la première connexion, SSH affiche une alerte concernant l'authenticité de l'hôte. Il faut répondre yes pour ajouter l'instance aux known_hosts. Une fois connecté, on se retrouve sur l'instance Ubuntu dans le répertoire /home/ubuntu.

Il est important de s'assurer que les permissions du fichier .pem sont correctement configurées (chmod 400) avant la connexion, sans quoi SSH refusera la clé.

Capture d'écran — Connexion SSH à l'instance EC2 :

```
vboxuser@RedaHote:~/Documents/Projet-Linux/terraform$ ssh -i ~/.ssh/terraform-ec2.pem ubuntu@35.180.24.77
The authenticity of host '35.180.24.77 (35.180.24.77)' can't be established.
ED25519 key fingerprint is SHA256:Esei7dykLqCHl0LCY5Y0c7do9d1mi+rCdfyFX0aely.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '35.180.24.77' (ED25519) to the list of known hosts.
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-1004-aws x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Mon May 18 23:41:12 UTC 2026

System load:  1.46           Temperature:   -273.1 C
Usage of /:   11.7% of 18.25GB Processes:    136
Memory usage: 46%           Users logged in: 0
Swap usage:   0%            IPv4 address for ens5: 172.31.6.45

Expanded Security Maintenance for Applications is not enabled.

Select Language N
```

Figure 5.8- Connexion SSH à l'instance EC2 déployée avec Terraform

Chapitre 6 :Pipeline CD — Déploiement Continu

Le fichier `deploy.yml`, placé dans le dossier `.github/workflows/`, définit le pipeline de déploiement continu de l'application. Il prend le relais automatiquement après la réussite du pipeline CI et se charge de construire les images Docker, de provisionner l'infrastructure AWS via Terraform, et de déployer l'application sur le serveur de production.

Ce pipeline est composé de deux jobs qui s'enchaînent séquentiellement : le premier construit et publie les images Docker sur Docker Hub, le second se connecte au serveur via SSH pour déployer la nouvelle version de l'application.

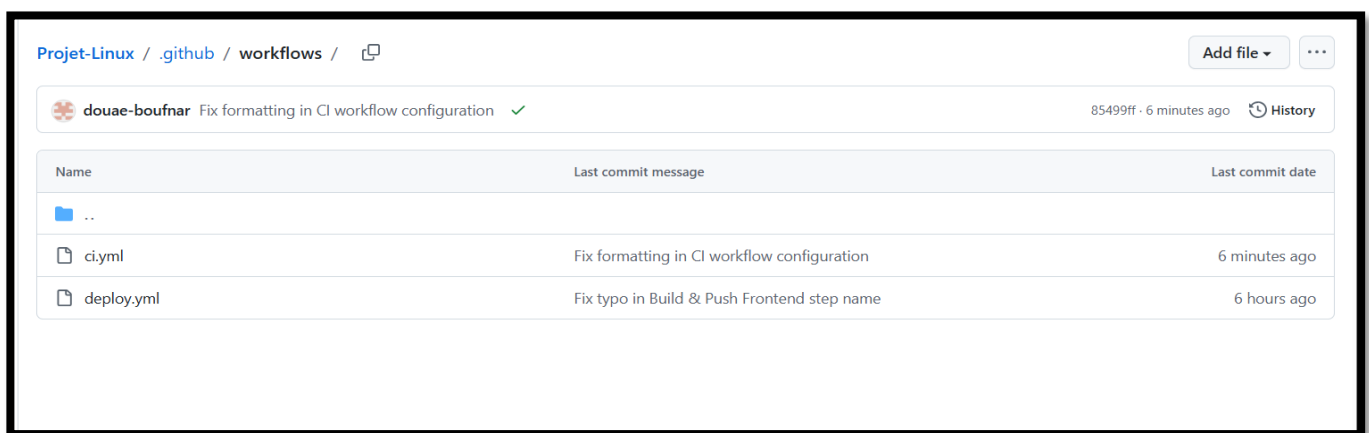


Figure 6.1- Structure des pipelines CI/CD du projet Projet-Linux sur GitHub

- Déclencheur :

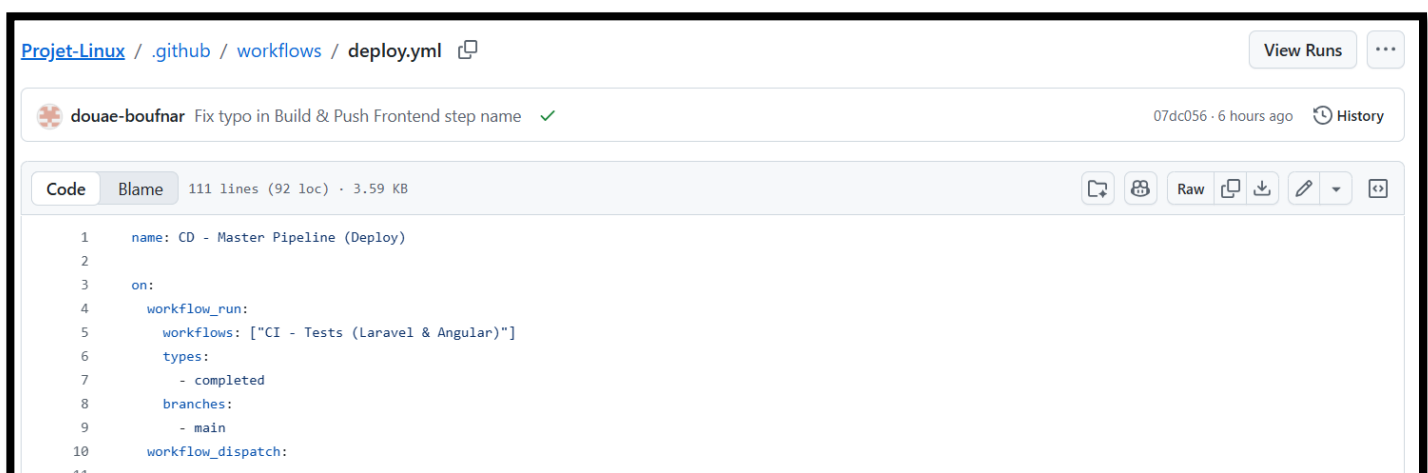


Figure 6.2- Workflow de déploiement continu (CD) conditionné par la réussite des tests CI

Le pipeline se déclenche automatiquement à la fin du pipeline CI si celui-ci a réussi, et uniquement sur la branche main. L'option `workflow_dispatch` permet également de le lancer manuellement depuis l'interface GitHub si besoin.

-Job 1 — Build & Push des images Docker

Ce job est conditionné par la réussite du pipeline CI grâce à la condition `if: workflow_run.conclusion == 'success'`. Il commence par définir une variable d'environnement `DOCKERHUB_REPO` contenant le nom du dépôt Docker Hub `reda7856/linux-project`, utilisée pour tagger les images.

Il récupère ensuite le code source, s'authentifie sur Docker Hub via les secrets GitHub, crée et configure le fichier `.env` du backend en adaptant l'hôte MySQL et le mot de passe pour l'environnement de production, puis construit l'image Docker du backend Laravel et la publie sur Docker Hub avec le tag `backend-latest`.

```

11
12   env:
13     DOCKERHUB_REPO: 'reda7856/linux-project'
14
15   jobs:
16     # 1. BUILD & PUSH DOCKER IMAGES
17     build-push:
18       name: Build & Push Docker Images
19       runs-on: ubuntu-latest
20       if: ${{ github.event.workflow_run.conclusion == 'success' || github.event_name == 'workflow_dispatch' }}
21       steps:
22         - name: Checkout
23           uses: actions/checkout@v4
24
25         - name: Login to Docker Hub
26           uses: docker/login-action@v3
27           with:
28             username: ${{ secrets.DOCKERHUB_USERNAME }}
29             password: ${{ secrets.DOCKERHUB_TOKEN }}
30
31         - name: Créer le fichier .env pour le backend
32           working-directory: ./mon-projet-backend
33           run: |
34             cp .env.example .env
35             sed -i 's|DB_HOST=127.0.0.1|DB_HOST=mysql|g' .env
36             sed -i 's|DB_PASSWORD=root|DB_PASSWORD=root|g' .env
37             echo "APP_KEY=base64:6W+deEHpcajwq/7ePWCLToMwXD3NPu9kQG+RNjdM3vA=" >> .env
38
39         - name: Build & Push Backend
40           uses: docker/build-push-action@v5
41           with:
42             context: ./mon-projet-backend

```

Figure 6.3- Workflow de déploiement continu

Build & Push de l'image Frontend

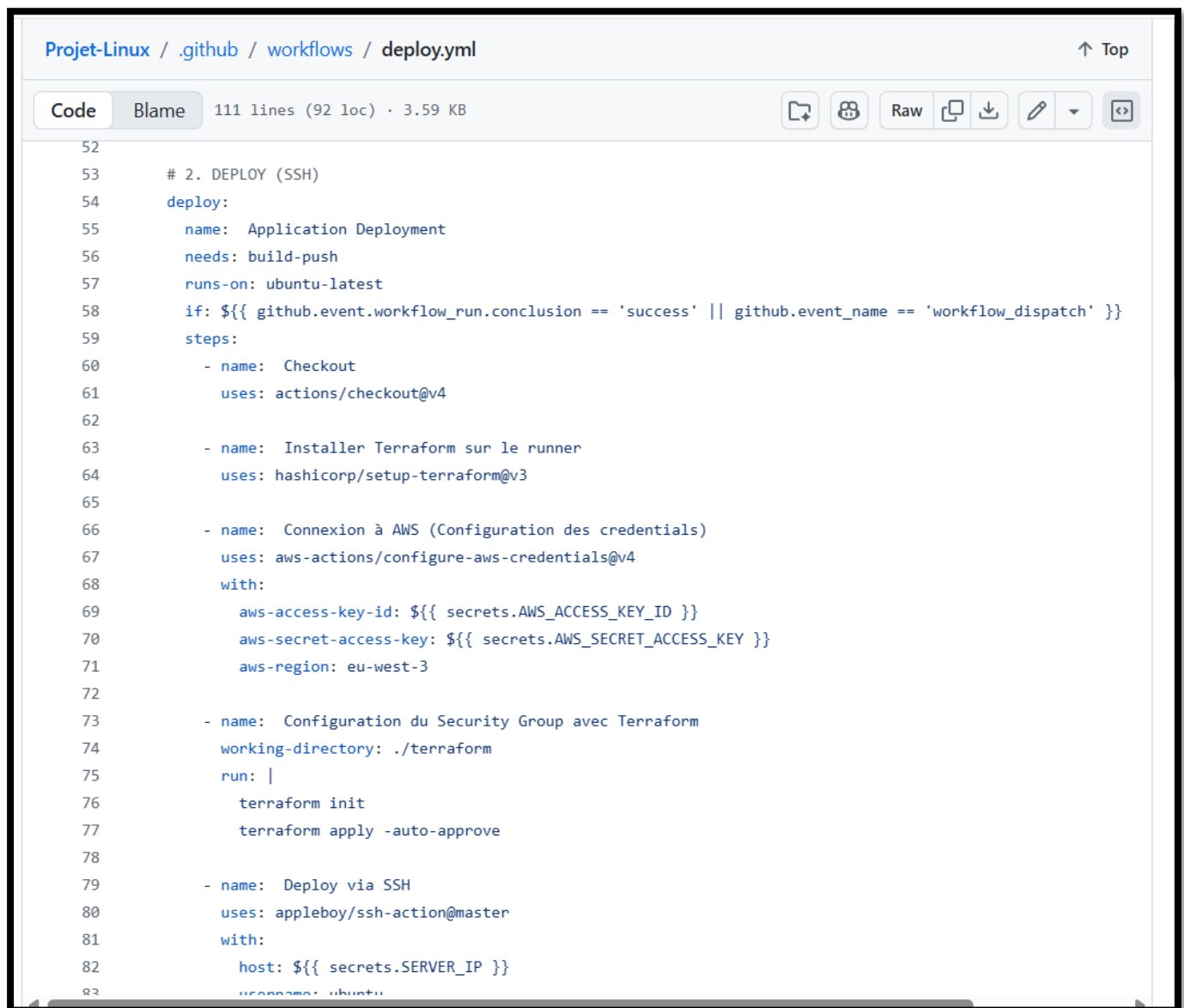
```
45      tags: ${{ env.DOCKERHUB_REPO }}:backend-latest
46
47      - name: Build & Push Frontend
48        uses: docker/build-push-action@v5
49        with:
50          context: ./frontend
51          push: true
52          tags: ${{ env.DOCKERHUB_REPO }}:frontend-latest
```

Figure 6.4- Configuration GitHub Actions pour le déploiement du frontend Angular sur Docker Hub

- Job 2 — Déploiement sur le serveur

Ce job s'exécute uniquement après la réussite du job précédent et se charge de provisionner l'infrastructure et de déployer la nouvelle version de l'application sur le serveur de production AWS.

- Installation de Terraform et configuration des credentials AWS pour accéder à l'infrastructure hébergée dans la région de Paris.
- Application automatique de la configuration du Security Group pour garantir que les règles de pare-feu sont à jour avant le déploiement.
- Connexion au serveur via SSH et récupération de la dernière version du code source.
- Mise à jour du fichier de configuration Docker pour utiliser les nouvelles images publiées sur Docker Hub.
- Redémarrage de tous les conteneurs ainsi que du serveur Nginx avec les nouvelles images.
- Réinitialisation de la base de données en appliquant les dernières modifications du fichier d'initialisation SQL.
- Nettoyage des anciennes images inutilisées pour libérer de l'espace disque sur le serveur.
- Affichage des logs des conteneurs pour vérifier que le déploiement s'est déroulé correctement.



```
52
53 # 2. DEPLOY (SSH)
54 deploy:
55   name: Application Deployment
56   needs: build-push
57   runs-on: ubuntu-latest
58   if: ${{ github.event.workflow_run.conclusion == 'success' || github.event_name == 'workflow_dispatch' }}
59   steps:
60     - name: Checkout
61       uses: actions/checkout@v4
62
63     - name: Installer Terraform sur le runner
64       uses: hashicorp/setup-terraform@v3
65
66     - name: Connexion à AWS (Configuration des credentials)
67       uses: aws-actions/configure-aws-credentials@v4
68       with:
69         aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
70         aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
71         aws-region: eu-west-3
72
73     - name: Configuration du Security Group avec Terraform
74       working-directory: ./terraform
75       run: |
76         terraform init
77         terraform apply -auto-approve
78
79     - name: Deploy via SSH
80       uses: appleboy/ssh-action@master
81       with:
82         host: ${{ secrets.SERVER_IP }}
83         username: ubuntu
```

Figure 6.5- Job de déploiement SSH avec Terraform et AWS dans le pipeline CD

The screenshot shows a GitHub Actions workflow file named `deploy.yml` located at `Projet-Linux / .github / workflows / deploy.yml`. The file contains 111 lines of YAML code. The workflow is triggered on a push to the `main` branch. It sets environment variables for `host`, `username`, and `key` using secrets. The `script` section includes commands to navigate to the project directory, reset the git repository, pull the latest changes, and use Docker Compose to pull and build the containers. It also includes commands to restart the `nginx` service, initialize the database, and display the status and logs of the containers.

```

82     host: ${ secrets.SERVER_IP }}
83     username: ubuntu
84     key: ${ secrets.SSH_PRIVATE_KEY }}
85     script: |
86         cd /home/ubuntu/projet
87         git reset --hard
88         git pull origin main
89
90         # Utiliser les nouvelles images
91         sed -i "s|backend-v1|backend-latest|g" docker-compose.yml
92         sed -i "s|Angular_v2|frontend-latest|g" docker-compose.yml
93
94         sudo docker compose pull
95         sudo docker compose up -d --build
96         sudo docker compose restart nginx
97
98         # Réinitialiser la base de données pour appliquer tous les changements de init.sql
99         sudo docker exec -i laravel_db mysql -u root -proot -e "DROP DATABASE IF EXISTS laravel_db; CREATE DATABASE laravel_db;"
100        sudo docker exec -i laravel_db mysql -u root -proot laravel_db < mysql/init.sql
101
102        sudo docker image prune -f
103
104        echo "=== STATUS DES CONTENEURS ==="
105        sudo docker compose ps
106
107        echo "=== LOGS DU BACKEND ==="
108        sudo docker compose logs backend --tail=100
109
110        echo "=== LOGS DE NGINX ==="
111        sudo docker compose logs nginx --tail=100

```

Figure 6.6- Étapes de déploiement automatisé

Démonstration de succès de CD :

The screenshot shows the GitHub Actions workflow runs history. It displays 119 workflow runs. The first two runs are highlighted:

Workflow	Event	Status	Branch	Actor	Time
CD - Master Pipeline (Deploy)	CD - Master Pipeline (Deploy) #26: completed by douae-boufnar	Completed	main	douae-boufnar	48 minutes ago 3m 1s
Fix formatting in CI workflow configuration	CI - Tests (Laravel & Angular) #93: Commit 85499ff pushed by douae-boufnar	Completed	main	douae-boufnar	49 minutes ago 23s

Figure 6.7- Historique des exécutions des workflows GitHub Actions

Tester l'ajout d'une catégorie :

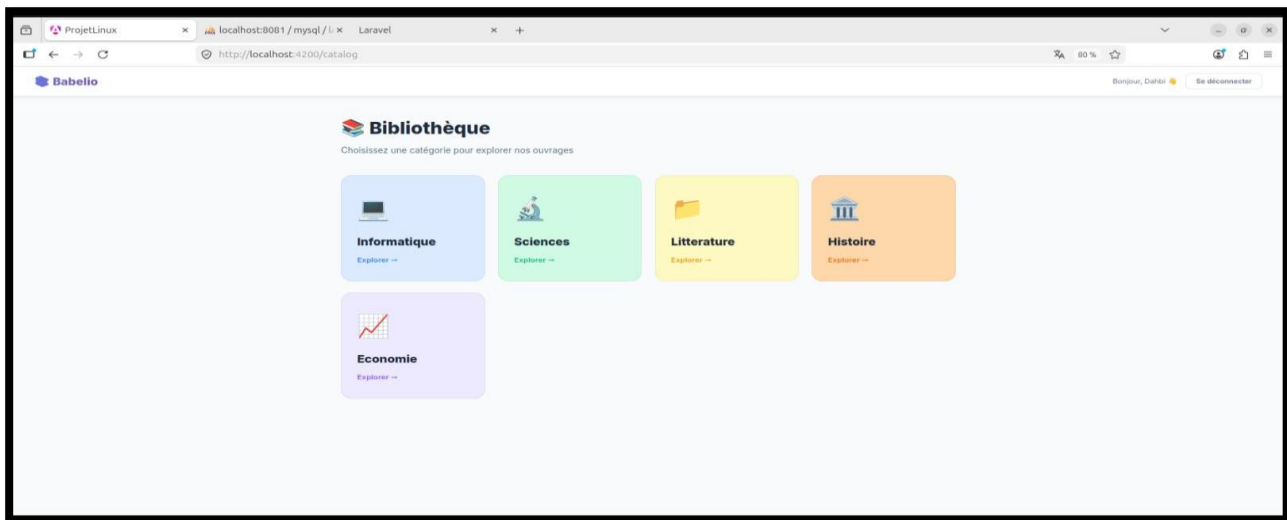


Figure 6.8- L'interface de l'application avant

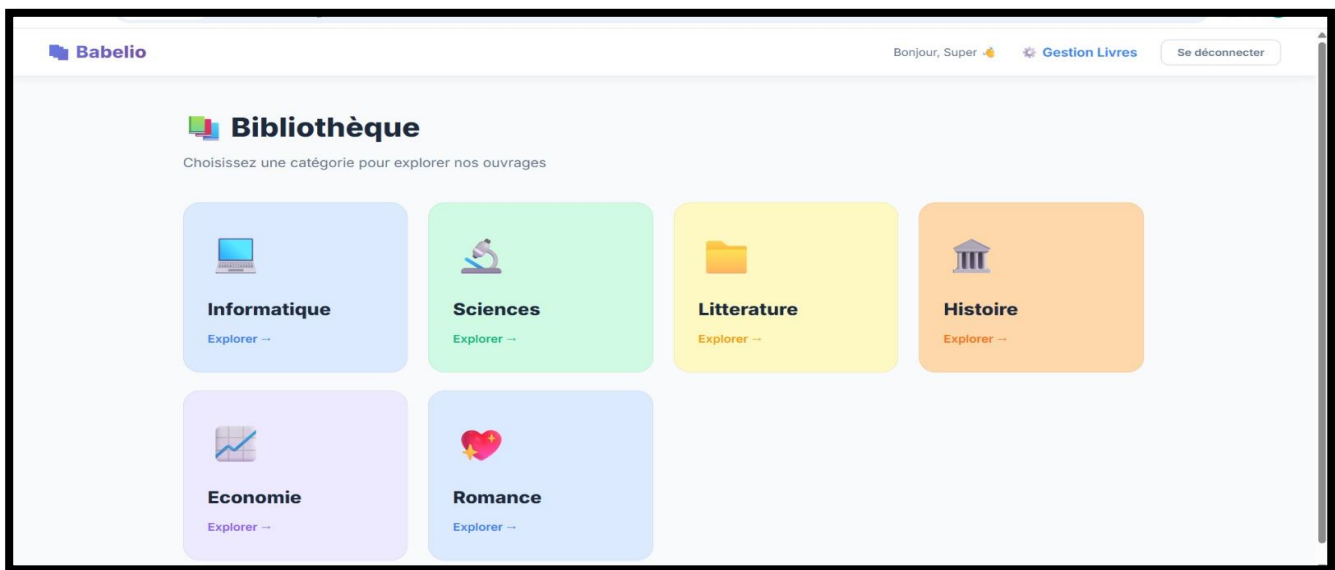


Figure 6.9- L'interface de l'application après le déploiement de la modification

Bénéfice du pipeline CD dans notre projet :

Le pipeline CD apporte une automatisation complète du déploiement, ce qui élimine toute intervention manuelle sur le serveur à chaque nouvelle version de l'application.

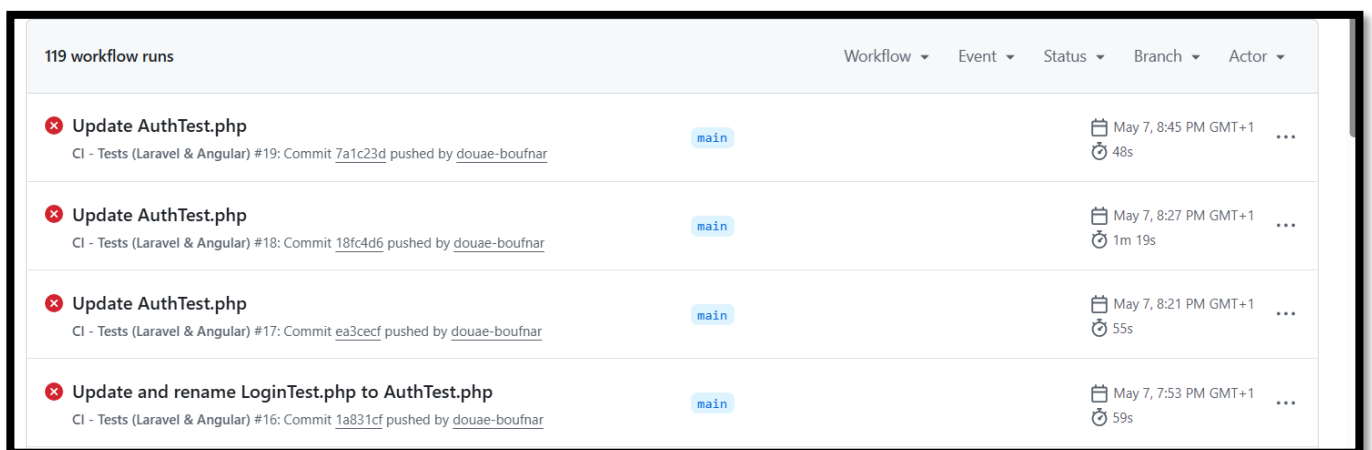
- Chaque modification validée par les tests est déployée automatiquement en production, réduisant considérablement le délai entre le développement et la mise en ligne.
- Les images Docker sont reconstruites et publiées systématiquement, garantissant que le serveur utilise toujours la version la plus récente du backend et du frontend.

- L'infrastructure AWS est provisionnée et mise à jour automatiquement via Terraform avant chaque déploiement, assurant une cohérence entre le code et l'environnement de production.
- La base de données est réinitialisée à chaque déploiement, évitant les conflits entre les anciennes et les nouvelles structures de données.
- L'ensemble du processus est tracé et visible dans l'historique GitHub Actions, offrant une transparence totale sur chaque déploiement effectué.

Chapitre 7 : Difficultés rencontrées durant le projet :

1)Échec des workflows GitHub Actions (CI/CD)

Lors des pushes sur la branche main, les pipelines CI/CD configurés avec GitHub Actions échouaient régulièrement, empêchant l'exécution des tests et le déploiement automatique de l'application. Les principales erreurs provenaient du backend Laravel, où le fichier .env et certaines dépendances n'étaient pas correctement configurés, ainsi que du frontend Angular, dont les tests ne fonctionnaient pas dans l'environnement GitHub Actions à cause de l'absence d'interface graphique et de problèmes de versions Node.js.



Workflow	Event	Status	Branch	Actor	Run Name	Commit	Pushed by	Time	Duration
Update AuthTest.php	CI - Tests (Laravel & Angular) #19	Failed	main	douae-boufnar	May 7, 8:45 PM GMT+1	7a1c23d	douae-boufnar	48s	...
Update AuthTest.php	CI - Tests (Laravel & Angular) #18	Failed	main	douae-boufnar	May 7, 8:27 PM GMT+1	18fc4d6	douae-boufnar	1m 19s	...
Update AuthTest.php	CI - Tests (Laravel & Angular) #17	Failed	main	douae-boufnar	May 7, 8:21 PM GMT+1	ea3cecf	douae-boufnar	55s	...
Update and rename LoginTest.php to AuthTest.php	CI - Tests (Laravel & Angular) #16	Failed	main	douae-boufnar	May 7, 7:53 PM GMT+1	1a831cf	douae-boufnar	59s	...

Figure7.1 - Liste des runs du pipeline qui sont échouées

Solution :

Pour résoudre ces problèmes, le pipeline CI a été amélioré en séparant les tests Laravel et Angular dans deux jobs distincts. L'environnement Laravel est désormais configuré automatiquement avec la génération du fichier .env et de la clé d'application. Les dépendances PHP et Node.js sont installées automatiquement avec les bonnes versions. Les tests Angular ont également été configurés en mode ChromeHeadless afin de fonctionner correctement dans GitHub Actions. Grâce à ces corrections, les pipelines CI/CD s'exécutent maintenant correctement et permettent le déploiement automatique de l'application.

2) problème lors de déploiement :

Lors du déploiement automatique via le pipeline CD, l'interface frontend Angular s'affichait correctement, mais une erreur Server Error apparaissait lors de la connexion utilisateur. Ce problème provenait principalement du backend Laravel après le redémarrage des conteneurs Docker. Les causes identifiées étaient liées à la reconfiguration automatique de la base de données, à l'absence de certaines données utilisateurs après l'exécution du fichier init.sql, ainsi qu'à des erreurs de configuration Laravel (APP_KEY, cache ou connexion MySQL). Ces problèmes empêchaient l'API backend de répondre correctement aux requêtes du frontend.

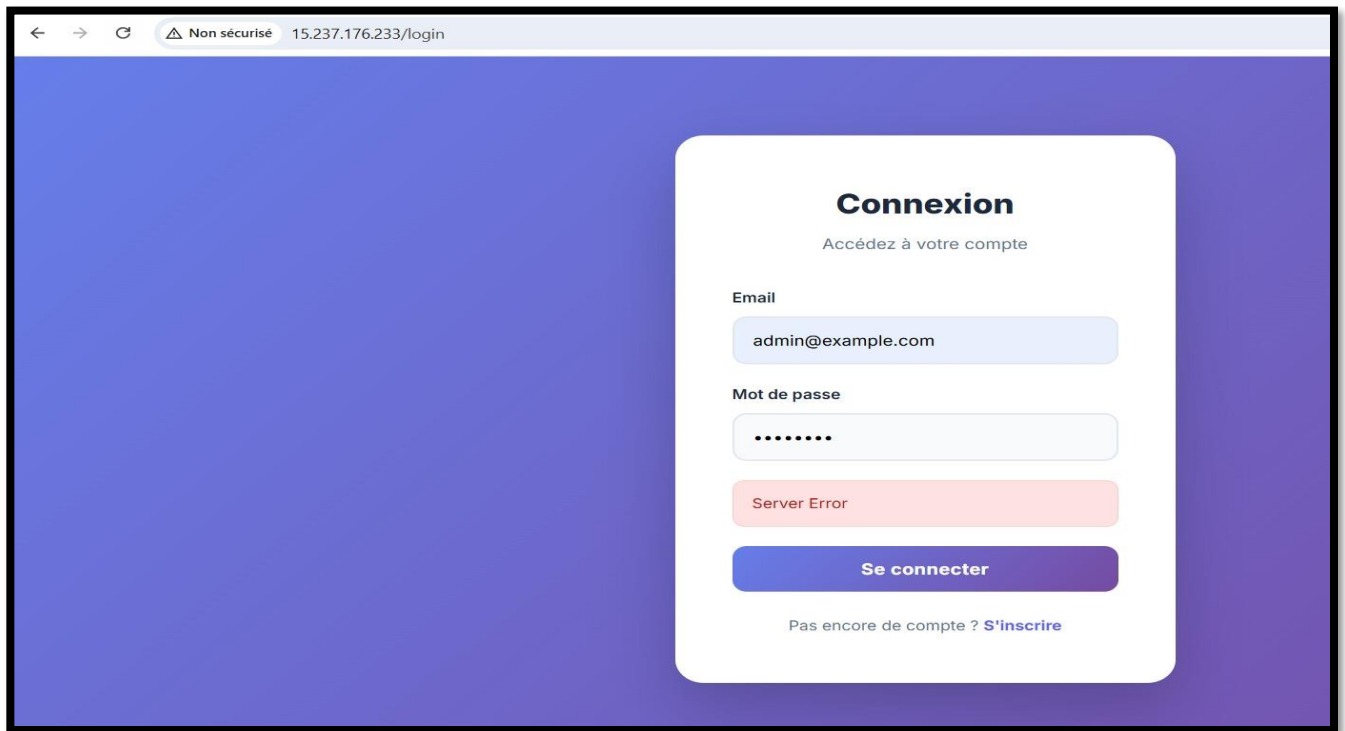


Figure 7.3 - Interface de connexion de l'application (page /login) avec tentative d'authentification

Solution :

Pour résoudre ce problème, plusieurs vérifications et corrections ont été effectuées sur le serveur après le déploiement. Les logs Docker du backend et de NGINX ont été analysés afin d'identifier les erreurs Laravel et MySQL. La configuration du fichier .env et la clé APP_KEY ont été corrigées, puis les caches Laravel ont été réinitialisés avec les commandes config:clear et cache:clear. La base de données a également été vérifiée afin de confirmer la présence des tables et des utilisateurs nécessaires à l'authentification. Enfin, le pipeline CD a été amélioré pour exécuter automatiquement les migrations et reconfigurer correctement l'environnement backend après chaque déploiement Docker.

3) Gestion des permissions et des logs dans l'environnement Docker

Au cours du développement du backend Laravel, nous avons rencontré des difficultés liées à la gestion des permissions des fichiers dans l'environnement Docker. Lors de l'exécution de l'application, Laravel ne pouvait pas écrire dans les fichiers de logs situés dans le dossier storage/logs, provoquant des erreurs serveur et l'arrêt de certaines fonctionnalités. Ce problème était principalement dû aux permissions Linux appliquées aux volumes Docker et aux fichiers générés automatiquement par le conteneur. La résolution de cette difficulté a nécessité une

meilleure configuration des droits d'accès (chmod, chown) ainsi qu'une bonne gestion des dossiers storage et bootstrap/cache.

4) Problèmes liés à GitHub

Durant le développement collaboratif du projet sur GitHub, chaque membre de l'équipe travaillait sur sa propre branche afin de développer ses fonctionnalités de manière indépendante. Cependant, lors de la fusion (merge) des branches vers la branche principale main, plusieurs conflits de code sont apparus. Ces conflits étaient principalement causés par des modifications simultanées des mêmes fichiers ou des mêmes sections de code, notamment dans les composants Angular, les fichiers de configuration et certaines fonctionnalités du backend Laravel. Ces problèmes compliquaient l'intégration du travail de l'équipe et pouvaient provoquer des erreurs dans les pipelines CI/CD.

Solution mise en place :

Pour résoudre ces problèmes, une meilleure organisation du travail collaboratif a été adoptée. Chaque développeur continuait à travailler sur une branche dédiée à sa fonctionnalité, puis les modifications étaient fusionnées progressivement dans la branche main après vérification. Avant chaque fusion, les membres mettaient à jour leurs branches avec la dernière version du projet afin de limiter les conflits. Lorsque des conflits apparaissaient, ils étaient résolus manuellement en analysant les différences entre les versions du code et en conservant les modifications correctes. Cette méthode a permis d'améliorer la stabilité du projet et de faciliter l'intégration continue.

Chapitre 8 : Retour d'expérience :

- AYA DAHBI

Ce projet m'a permis d'acquérir une expérience importante en tant que chef de projet et responsable de la partie Docker. J'ai participé à la création et la configuration des fichiers Dockerfile et Docker Compose afin de standardiser l'environnement de développement, tout en assurant la répartition et le suivi des tâches entre les membres de l'équipe. Cette expérience m'a aidé à développer mes compétences techniques en DevOps ainsi que mes capacités d'organisation et de gestion d'équipe.

- BOUFNAR DOUAE

En tant que responsable GitHub et CI/CD, mon rôle principal était de gérer le versionnement du code, l'organisation des branches et le suivi des pull requests afin d'assurer une collaboration efficace au sein de l'équipe. J'étais également chargé de la mise en place et du maintien des pipelines CI/CD, intégrant Terraform pour automatiser le provisionnement de l'infrastructure et le déploiement des applications. Grâce au processus de déploiement continu (CD), chaque modification du code pouvait être automatiquement buildée, testée et déployée directement sur les environnements, permettant des mises à jour rapides, fiables et avec un minimum d'intervention manuelle.

- ELASRI FATIMA ZAHRA

L'intégration de Docker dans ce projet a grandement amélioré l'environnement de développement en assurant une isolation et une standardisation complètes des services backend. Grâce aux fichiers Dockerfile et Docker Compose, l'application Laravel, la base de données MySQL et les autres services pouvaient être lancés et configurés rapidement dans un environnement identique pour tous les membres de l'équipe. Cette approche a permis de réduire les problèmes de compatibilité, de simplifier le déploiement et de faciliter le travail collaboratif. En tant que responsable Docker et chef de projet, cette expérience m'a permis de renforcer mes compétences en conteneurisation, en gestion d'infrastructure et en organisation d'équipe dans un contexte DevOps moderne.

- ZAKARIA REDA

Ce projet m'a permis d'approfondir mes compétences en conteneurisation avec Docker, en configurant des images personnalisées et en orchestrant plusieurs services via Docker Compose. J'ai également pu mettre en pratique l'architecture MVC à travers le développement du backend Laravel et du frontend Angular, en comprenant concrètement comment ces deux couches communiquent via une API. Par ailleurs, la mise en place de l'infrastructure avec Terraform m'a initié aux principes de l'Infrastructure as Code et au déploiement automatisé sur le cloud AWS. Ce projet m'a ainsi offert une vision complète du cycle de développement d'une application web moderne, de l'environnement local jusqu'à la mise en production sur un serveur cloud.

-OUGGOURNI AYMEN

Le développement du frontend avec Angular a été une expérience très enrichissante qui m'a permis de découvrir son architecture modulaire basée sur les composants, services et modèles, ainsi que la communication entre Angular et Laravel à travers les API REST. J'ai acquis des compétences en data binding, routing et organisation d'un projet frontend structuré. L'utilisation de Docker, GitHub et des pratiques DevOps m'a également aidé à mieux comprendre la conteneurisation, le déploiement moderne des applications web et le travail collaboratif en équipe.

Conclusion :

En conclusion, ce projet nous a permis de concevoir et de déployer une bibliothèque numérique moderne tout en appliquant concrètement les principes du DevOps et du développement collaboratif. Grâce à Docker et Docker Compose, nous avons réussi à créer un environnement de développement standardisé et cohérent pour tous les membres de l'équipe, facilitant ainsi l'intégration et la communication entre les différents services de l'application. L'intégration de GitHub Actions et des pipelines CI/CD nous a permis d'automatiser les tests, le build et le déploiement de l'application, tandis que Terraform et AWS ont assuré l'automatisation de l'infrastructure cloud et du déploiement sur un serveur Ubuntu Linux.

Ce projet nous a également permis de renforcer nos compétences techniques en Laravel, Angular, MySQL, Nginx et Linux, tout en découvrant les pratiques modernes de conteneurisation, d'intégration continue et d'automatisation. Au-delà des aspects techniques, cette expérience a développé notre capacité à travailler en équipe, à gérer un projet collaboratif et à résoudre des problématiques réelles dans un environnement proche du monde professionnel. Ainsi, ce projet représente une expérience complète et formatrice dans le domaine du développement et du déploiement d'applications web modernes

Annexes :

- Lien GitHub du projet : <https://github.com/douae-boufnar/Projet-Linux.git>
- Lien de repository dans dockerhub :
<https://hub.docker.com/repository/docker/reda7856/linux-project/general>

